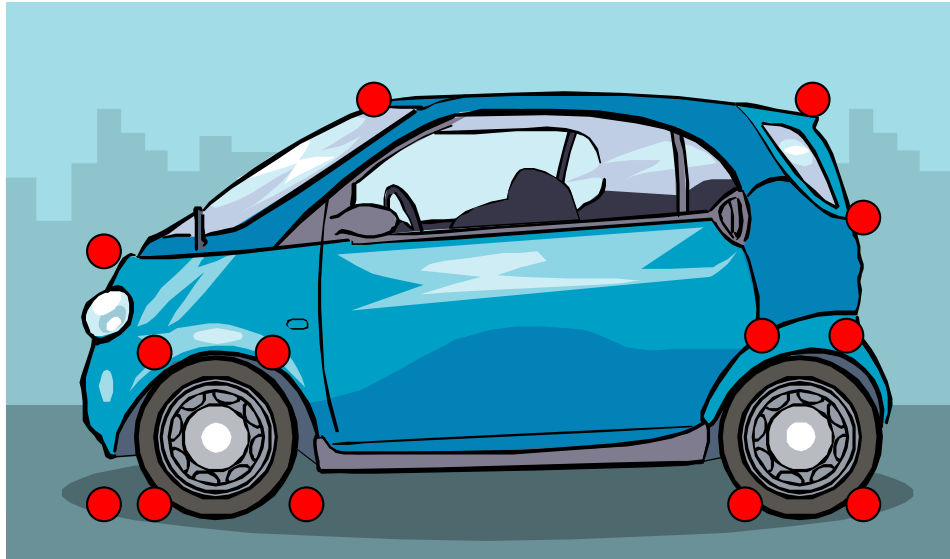


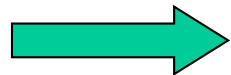
Le Curve B-Spline e NURBS

Computer Graphics 2025-2026

PROBLEMA: disegno del profilo dell'auto



Se fissiamo 13 punti

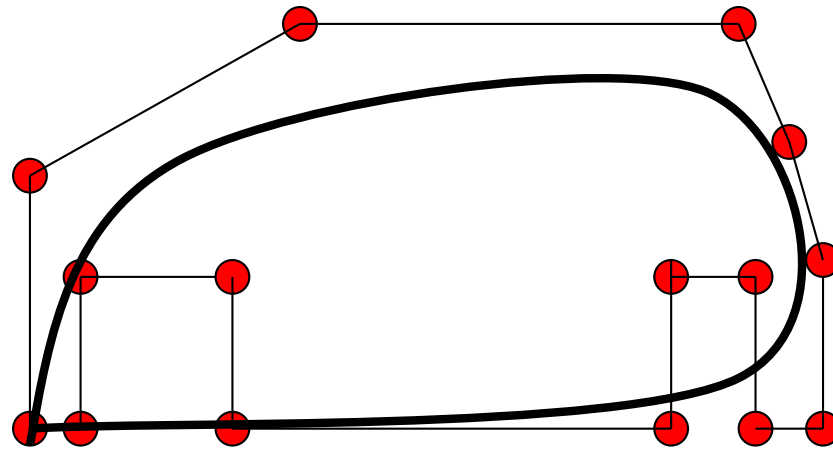


La curva di Bezier ha grado 12

La curva interpola solo il primo e l'ultimo punto

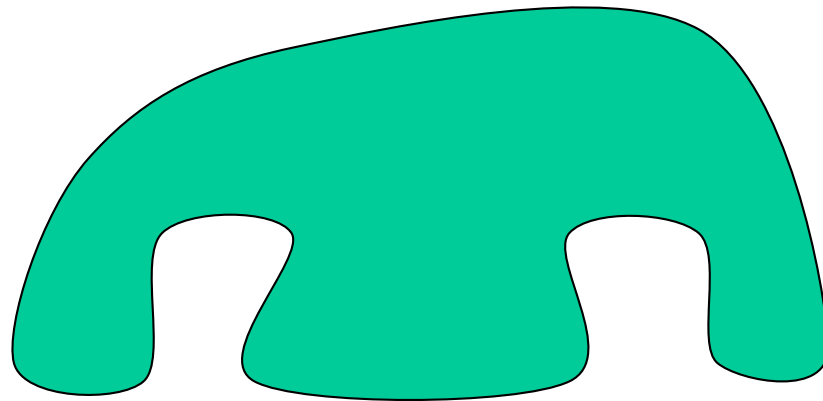
La curva giace all'interno del convex hull

Curva di Bezier di grado 12



Risultato completamente insoddisfacente!

Curva di Bezier di grado 165 (166 punti di controllo!)



Risultato completamente insoddisfacente!

Dalle curve di Bezier alle curve (B-)spline

una delle maggiori difficoltà nelle applicazioni delle curve di Bezier è il loro essere "globali" :

modificando un punto si modifica tutta la curva
(no modifiche "locali")



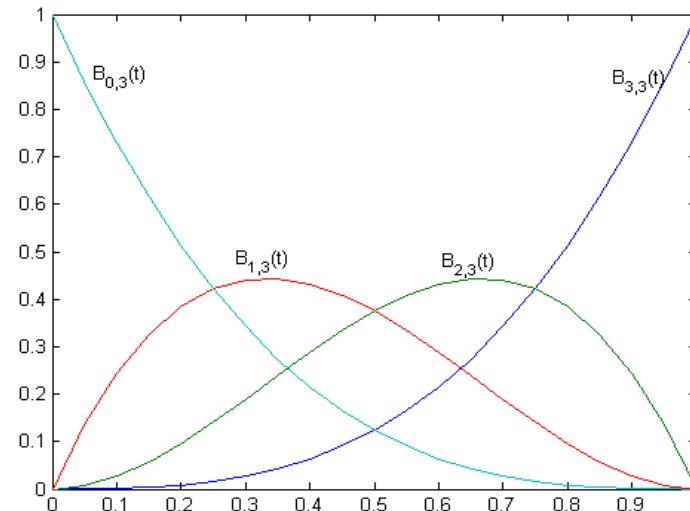
Le funzioni di base sono definite e sono non nulle in tutto l'intervallo di definizione della curva

Dalle curve di Bezier alle curve (B-)spline

una delle maggiori difficoltà nelle applicazioni delle curve di Bezier è il loro essere "globali" :

modificando un punto si modifica tutta la curva
(no modifiche "locali")

$$C(t) = \sum_{i=0}^n P_i B_{i,n}(t) \quad t \in [0,1]$$



Dalle curve di Bezier alle curve (B-)spline

una delle maggiori difficoltà nelle applicazioni delle curve di Bezier è il loro essere "globali" :

modificando un punto si modifica tutta la curva
(no modifiche "locali")



Abbiamo bisogno di una base
a supporto locale

Dalle curve di Bezier alle curve (B-)spline

una delle maggiori difficoltà nelle applicazioni delle curve di Bezier è il loro essere "globali" :

modificando un punto si modifica tutta la curva
(no modifiche "locali")



introduzione di un modello

A TRATTI

interpolante /approssimante

FUNZIONI POLINOMIALI A TRATTI INTERPOLANTI

Si ottengono suddividendo l'intervallo di definizione $[t_1, t_n]$ in sottointervalli con estremi coincidenti e costruendo su ciascun sottoinsieme il relativo polinomio interpolante.



Per ogni intervallo I_i si costruisce il polinomio interpolante di grado m .

Come costruire un
polinomio interpolante
a tratti che sia sufficientemente
regolare ?

Un compromesso fra le due esigenze:

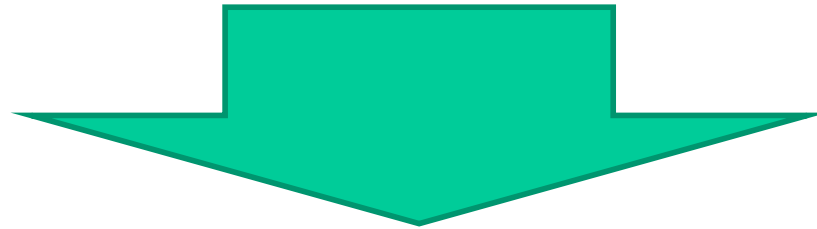
- Polinomi di grado moderatamente basso
- Sufficiente regolarità dell'intera funzione risultante (curva “smooth”)

è realizzato dalle

Funzioni Spline

Le funzioni B-spline

Le B-spline (basis spline) sono funzioni linearmente indipendenti



$N_{i,h}(t), \quad i=1,2,3,\dots \quad h = \text{grado}$

$\{N_{0,h}(t), \dots, N_{n,h}(t)\}$

base per lo spazio delle spline di grado h sul vettore dei nodi

Con le funzioni B-spline
entrano in gioco
i nodi
che svolgono un ruolo significativo

Sia $T = (t_0, t_1, \dots)$ il vettore dei nodi

Il Supporto locale delle B-spline

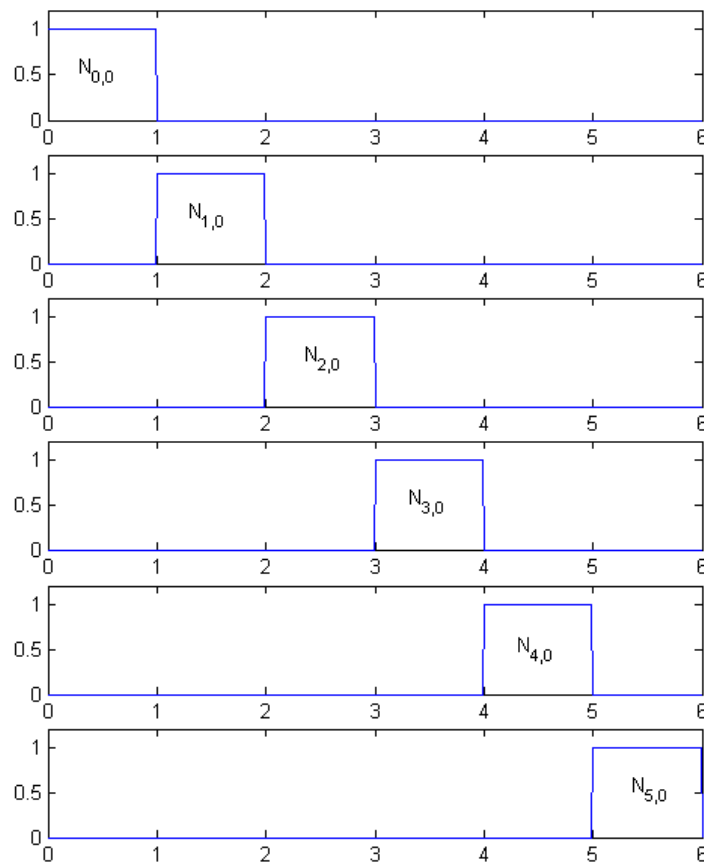
- La funzione B-spline di grado 0 $N_{i,0}(t)$
ha **supporto compatto** nell'intervallo $[t_i \ t_{i+1}]$
- la funzione B-spline di grado 1 $N_{i,1}(t)$
ha **supporto compatto** nell'intervallo $[t_i \ t_{i+2}]$
- ...
- La funzione B-spline di grado h $N_{i,h}(t)$
ha **supporto compatto** nell'intervallo $[t_i \ t_{i+h+1}]$

Supporto: insieme dei punti del dominio dove la funzione non si annulla.

Esempio: funzioni B-Spline di grado 0

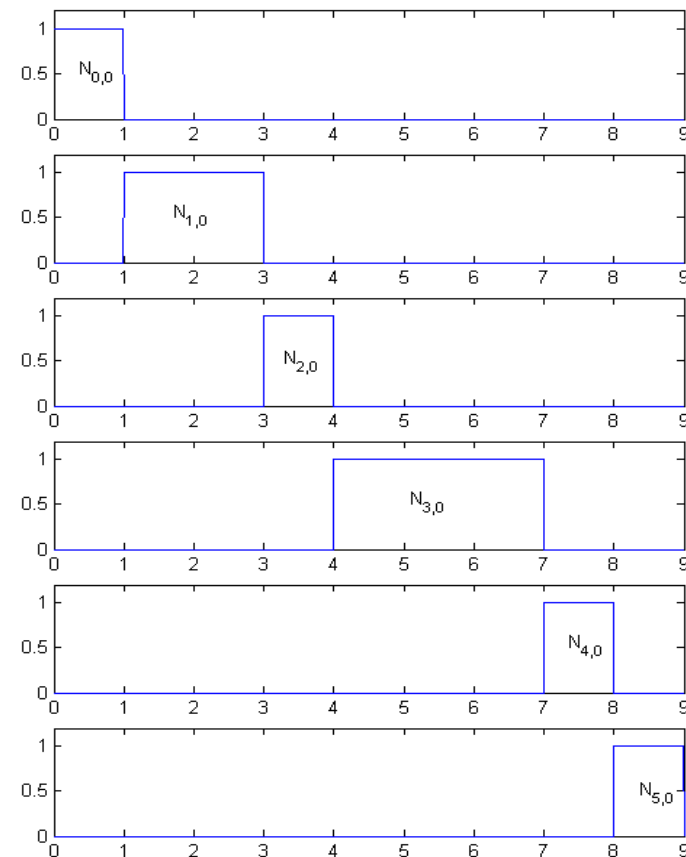
$T=(0,1,2,3,4,5,6)$

uniforme



$T=(0,1,3,4,7,8,9)$

non uniforme

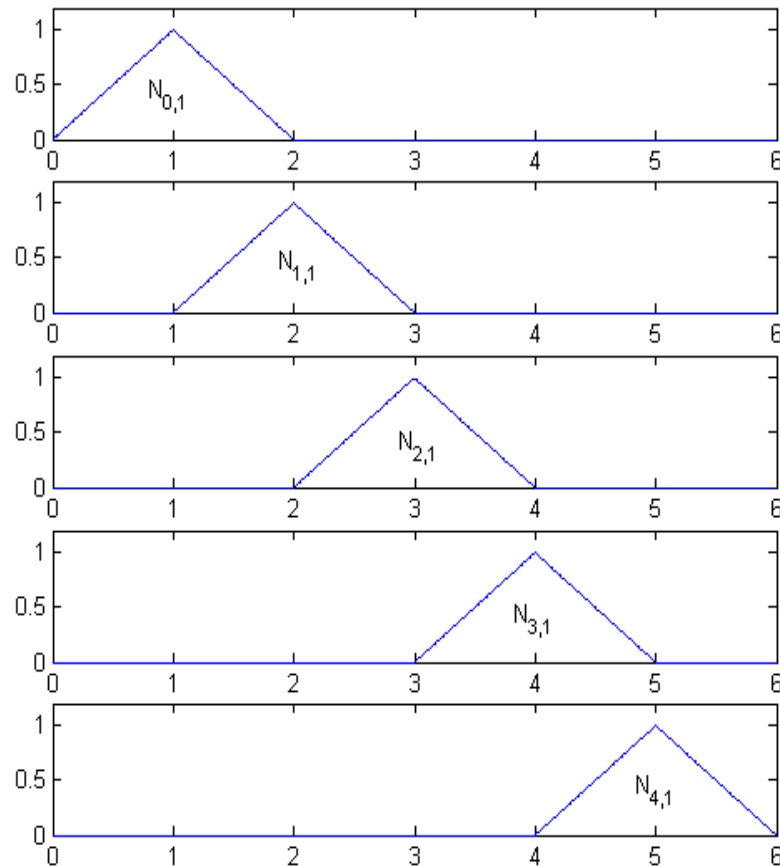


Non nulle nell'intervallo $[t_i, t_{i+h+1}]$

Esempio: funzioni B-Spline di grado 1

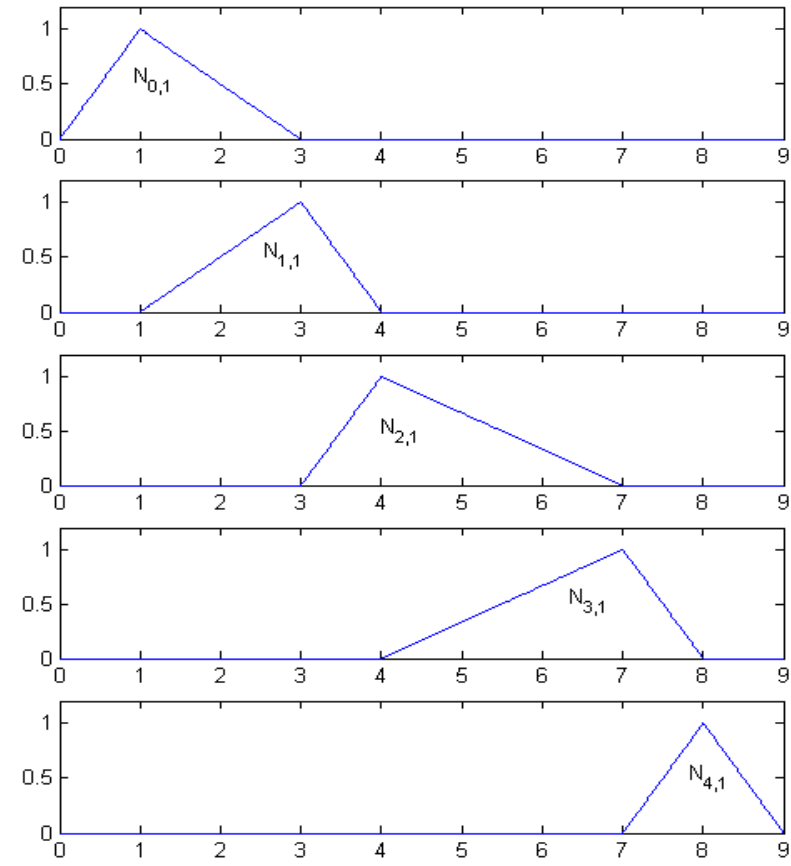
$T=(0, 1, 2, 3, 4, 5, 6)$

uniforme



$T=(0, 1, 3, 4, 7, 8, 9)$

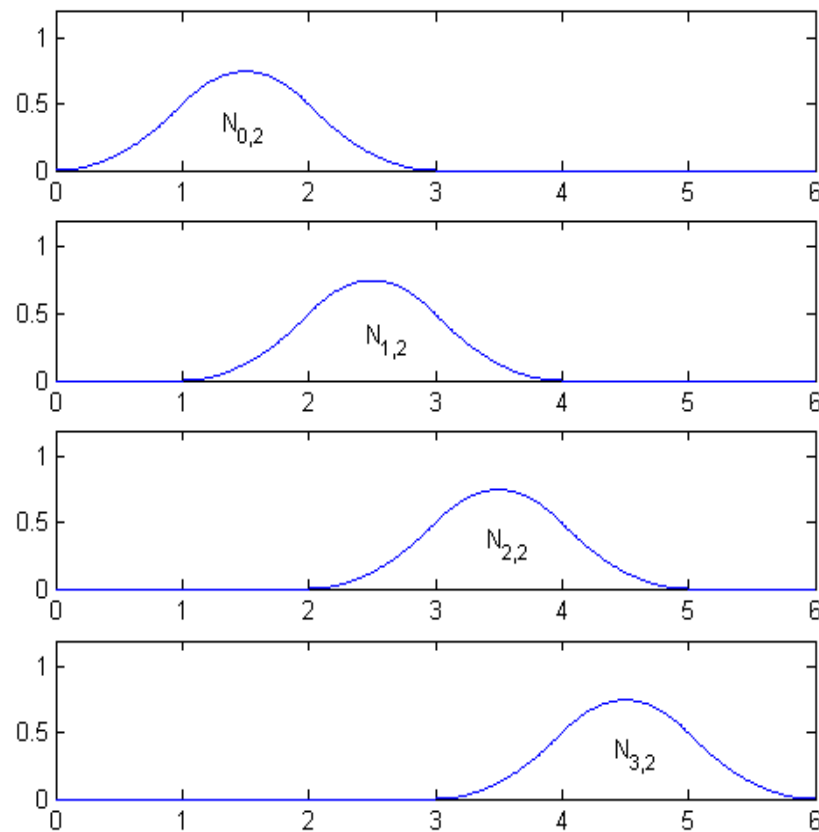
non uniforme



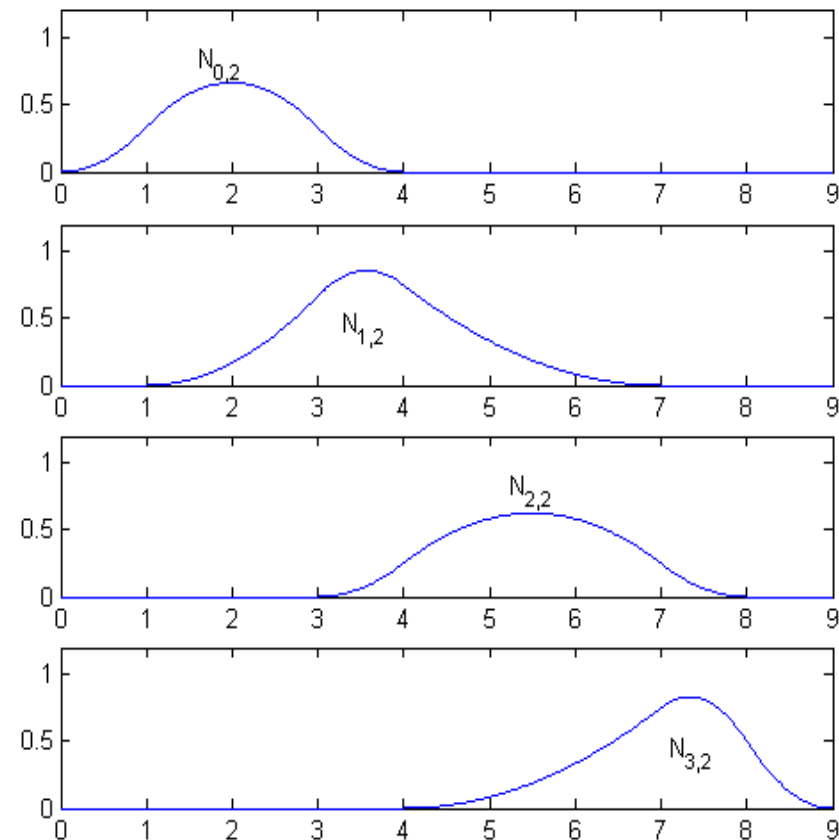
Non nulle nell'intervallo $[t_i, t_{i+h+1}]$, massimo in t_{i+1}

Esempio: funzioni B-Spline di grado 2

$T=(0, 1, 2, 3, 4, 5, 6)$
uniforme



$T=(0, 1, 3, 4, 7, 8, 9)$
non uniforme



Non nulle nell'intervallo $[t_i, t_{i+h+1}]$, massimo tra t_{i+1} e t_{i+2}

Le *curve* B-spline

$$C(t) = \sum_{i=0}^n P_i N_{i,h}(t)$$

Curva di grado h

Punti di controllo

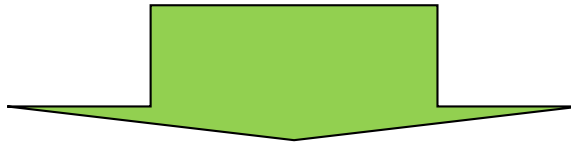
Funzione B-spline di grado h
definita in $t \in [t_i, t_{i+h+1}]$

nodi

Parametri di una curva B-Spline

- *n+1 punti di controllo* $P_0, \dots, P_n$
- *grado* h
- *vettore dei nodi* :

$$T = (t_0, t_1, \dots, t_{n+h+1}), \quad t_i \leq t_{i+1}$$



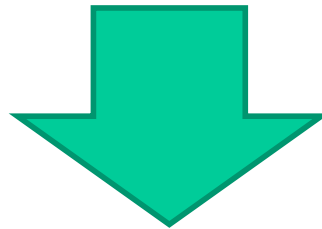
$$\# \text{ nodi} = \# \text{ punti} + \text{grado} + 1$$

$$n+h+2 \quad n+1 \quad h$$

Premesse completamente diverse da quelle alla base delle curve di Bezier!

Le *curve* B-spline e i nodi

Una curva B-spline è costituita da $n+1$ funzioni di base di grado h



Ciascuna funzione di base ($i = 0, n$) si costruisce per

$$t \in [t_i, t_{i+h+1}]$$



$$t_0, \dots, t_{n+h+1}$$

Curve B-Spline

- $P_0, \dots, P_n$ punti di controllo

$$C(t) = \sum_{i=0}^n P_i N_{i,h}(t) \quad t \in [t_h, t_{n+1}]$$

In t abbiamo bisogno di $h+1$ funzioni b-spline non nulle

con $N_{i,h}(t)$ *funzioni di base B-spline* di grado h

$$N_{i,h}(t) = \frac{t - t_i}{t_{i+h} - t_i} N_{i,h-1}(t) + \frac{t_{i+h+1} - t}{t_{i+h+1} - t_{i+1}} N_{i+1,h-1}(t)$$

Formula di Cox-de Boor

$$N_{i,0}(t) = \begin{cases} 1 & t_i \leq t < t_{i+1} \\ 0 & \text{altrimenti} \end{cases}$$

$$\begin{pmatrix} 0 \\ 0 \end{pmatrix} = 0$$

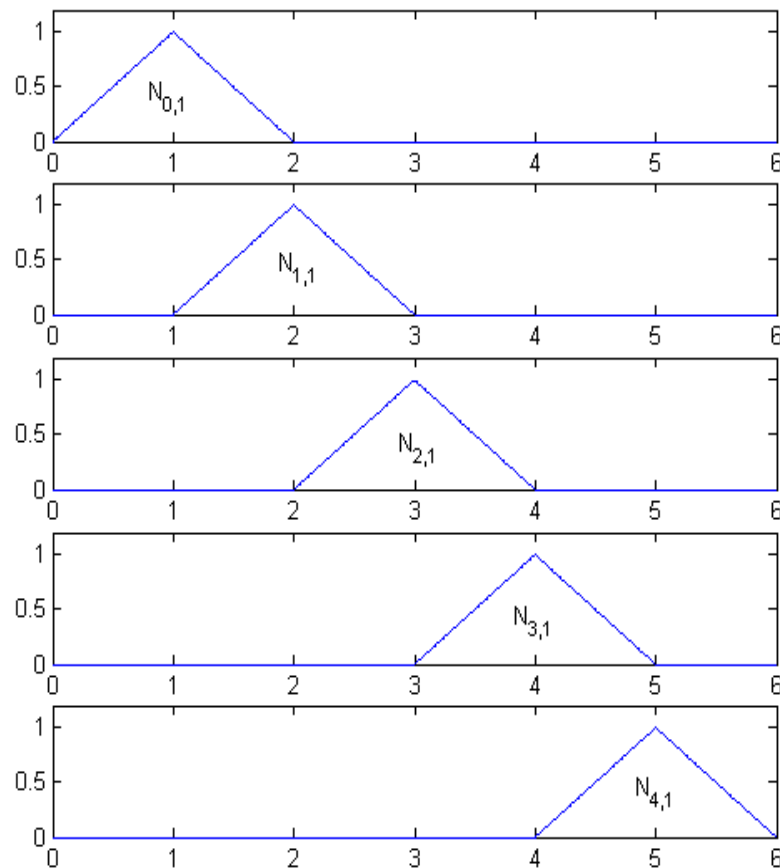
e *vettore dei nodi* $T = (t_0, t_1, \dots, t_{n+h+1})$, con

$$t_0 \leq t_1 \leq \dots \leq t_{n+h+1}$$

Composizione di funzioni B-Spline di grado 1

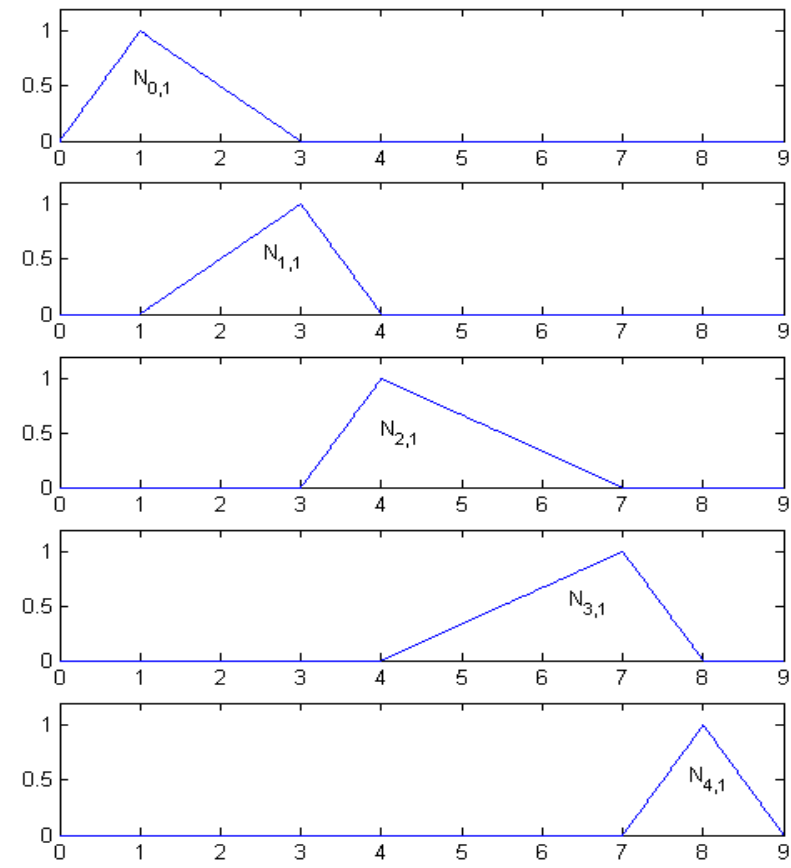
$T=(0, 1, 2, 3, 4, 5, 6)$

uniforme



$T=(0, 1, 3, 4, 7, 8, 9)$

non uniforme

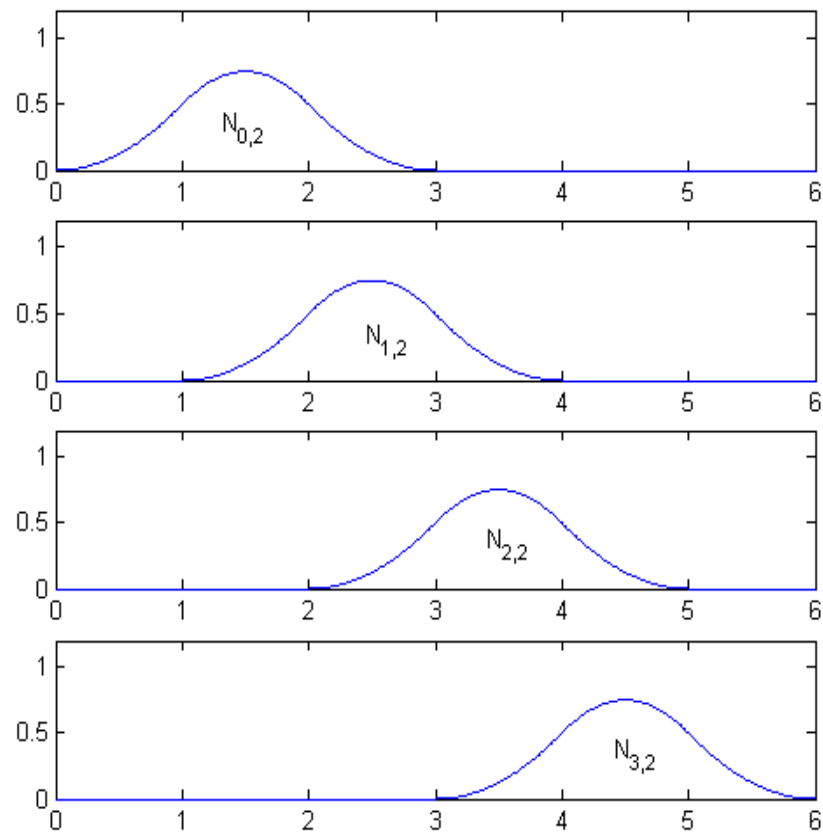


DOMINIO

Composizione di funzioni B-Spline di grado 2

$T=(0, 1, 2, 3, 4, 5, 6)$

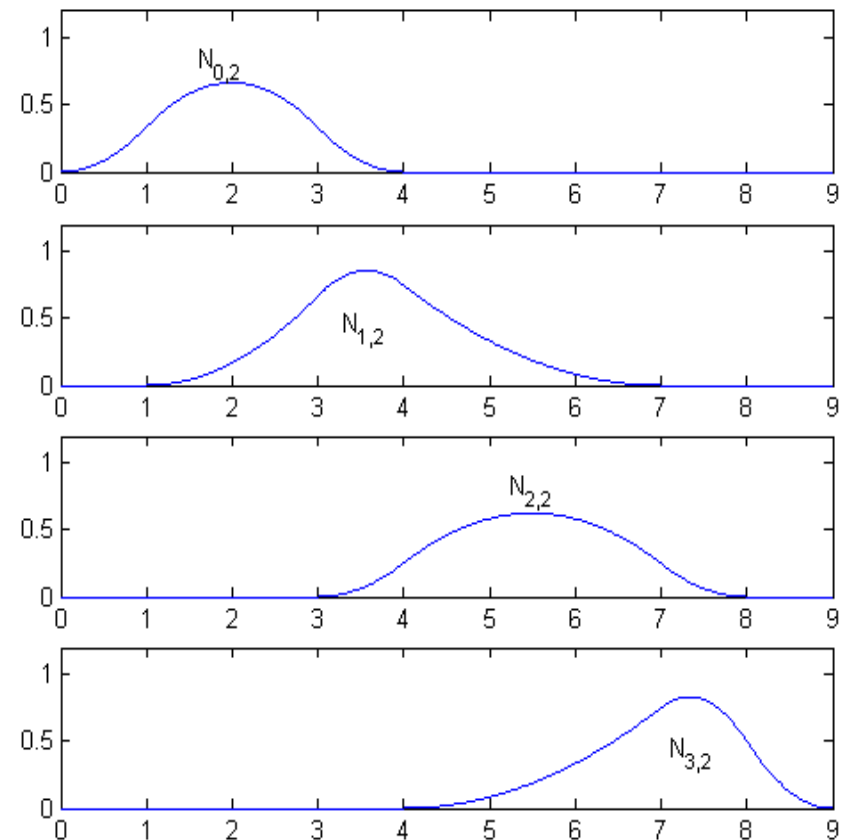
uniforme



DOMINIO

$T=(0, 1, 3, 4, 7, 8, 9)$

non uniforme

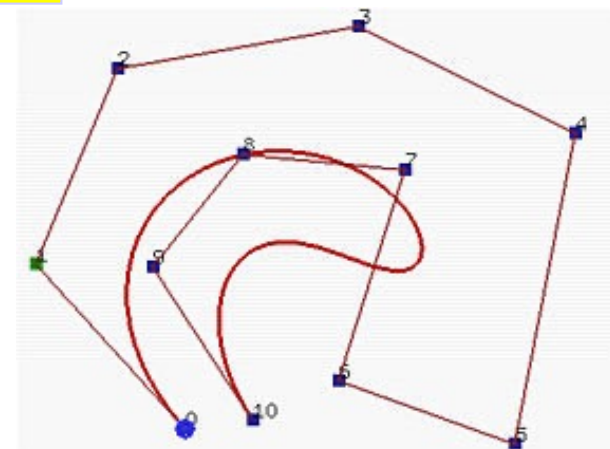
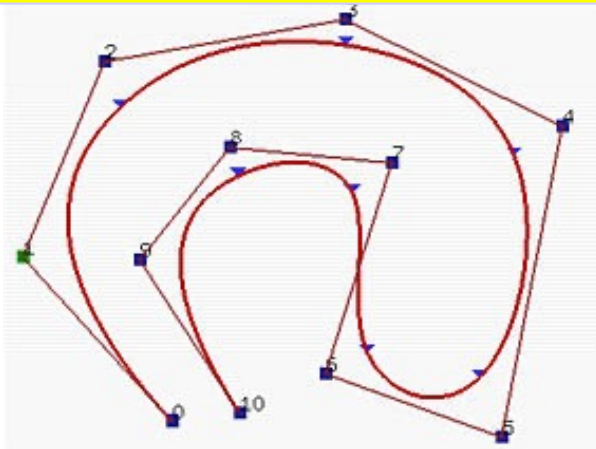


Le curve B-spline e le curve di Bezier

Curva B-spline : $h=3$ grado

$n+1 = 11$ punti di controllo

$n+h+2 = 15$ nodi



Curva di Bezier $h=10$ grado

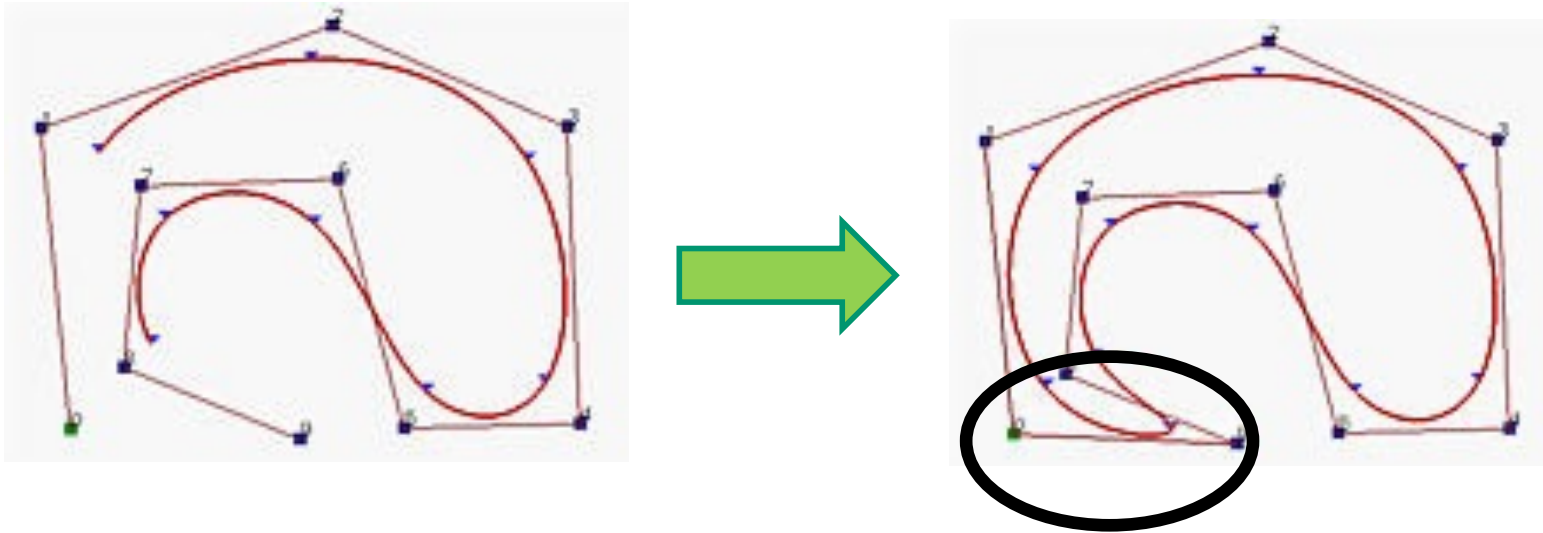
$n+1 = 11$ punti di controllo

controllo

Proprietà delle curve B-Spline

- In generale la curva B-spline **non** è chiusa
- Può essere *chiusa* se

l'ultimo punto di controllo P_n coincide con il primo P_0



Curva B-spline di grado 3

Proprietà delle curve B-Spline

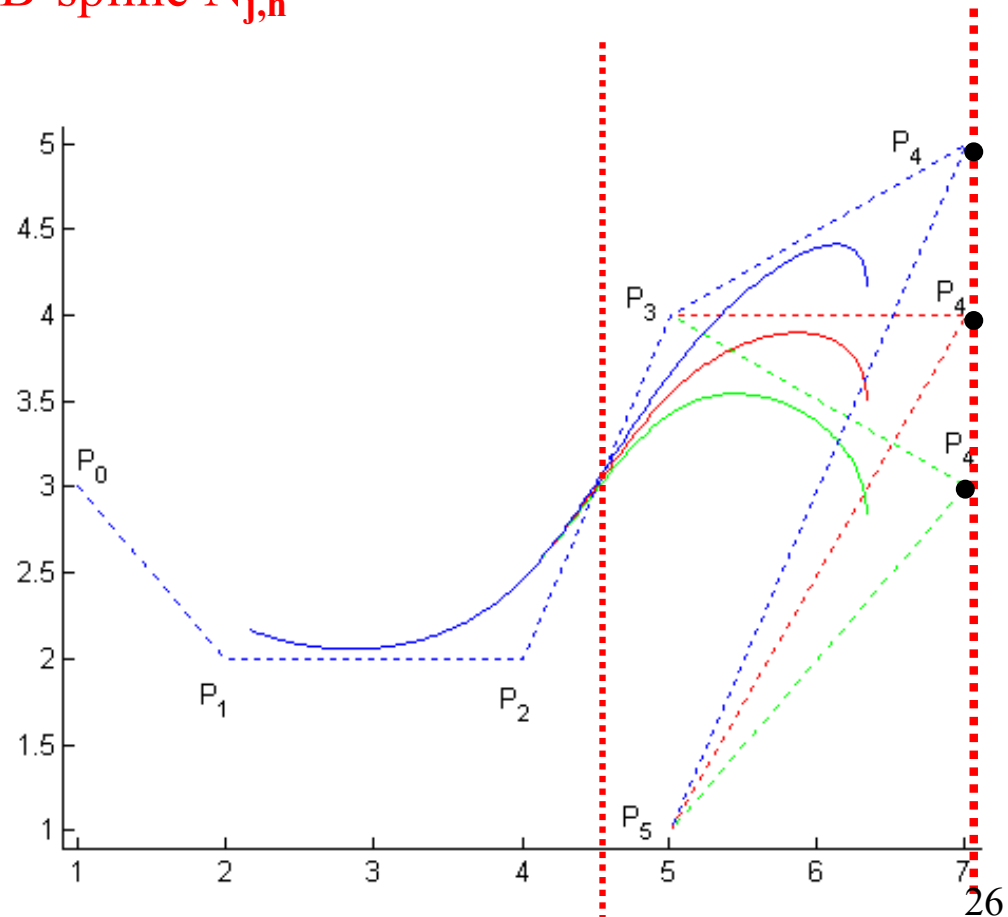
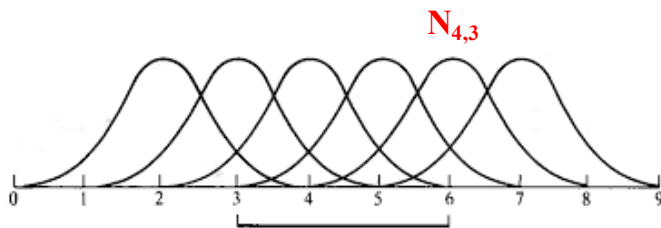
- *Controllo locale*: come conseguenza del supporto compatto delle B-spline, se P_j cambia, $C(t)$ risulta modificata solo in corrispondenza dei punti di definizione della funzione B-spline $N_{j,h}$

Esempio:

$h=3$:

$j=4$

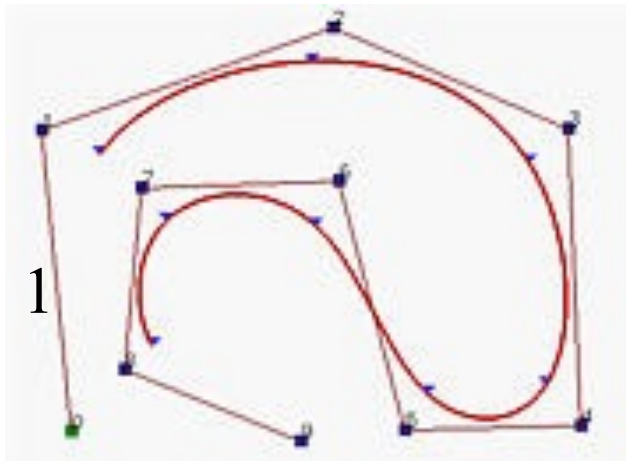
$$C(t) = \sum_{i=0}^n P_i N_{i,h}(t) \quad t \in [t_h, t_{n+1}]$$



Proprietà delle curve B-Spline

in generale la curva **non interpola**
alcun punto di controllo

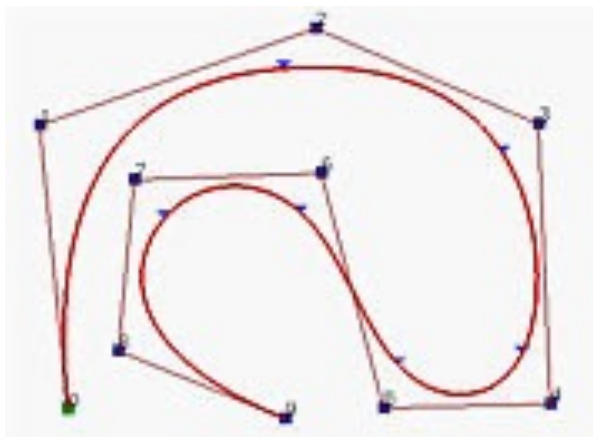
Curva B-spline di grado 3



- $10 = n+1$ punti di controllo ($n=9$)
- $h = 3$.
- $m = 13$ (14 nodi)

Proprietà delle curve B-Spline

Possiamo *modificare* la curva in modo tale che sia tangente al primo e all'ultimo segmento nel primo e nell'ultimo punto di controllo, rispettivamente. Per far ciò, il primo e l'ultimo nodo devono avere molteplicità h ($=$ grado) $+1$.



- i primi $h+1 = 4$ e gli ultimi 4 nodi devono essere identici.
- I restanti $14 - (4 + 4) = 6$ nodi possono essere ovunque nel dominio.

Curva B-spline di grado 3

Nodi estremi

$h=2$:

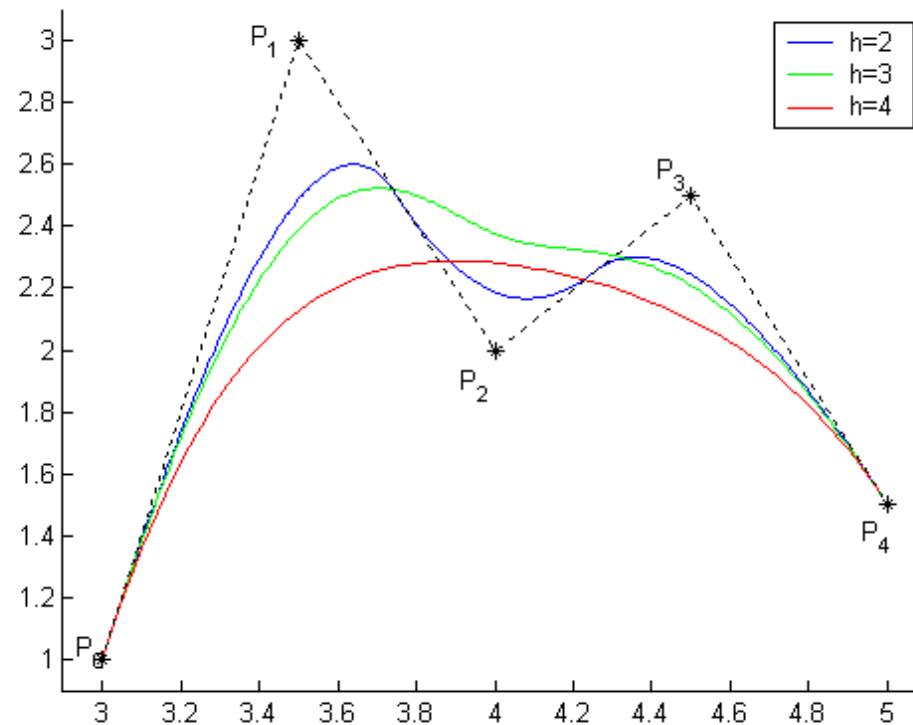
$$T=(\underline{0,0,0,1,2,3,3,3})$$

$h=3$:

$$T=(\underline{0,0,0,0,1,2,2,2,2})$$

$h=4$:

$$T=(\underline{0,0,0,0,0,1,1,1,1,1})$$



In generale:

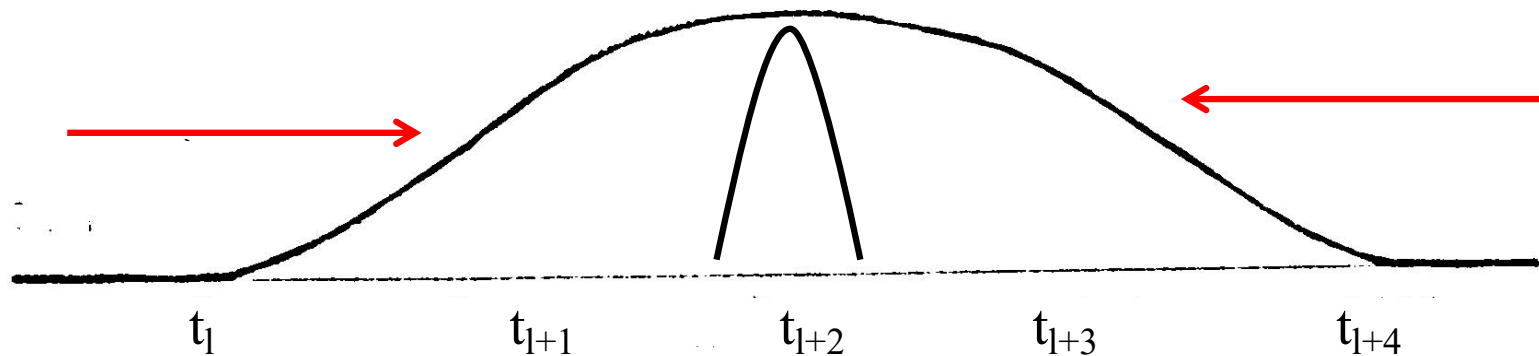
Fissato il grado h , all'aumentare della molteplicità ($molt$) di un nodo

La curva si avvicina al nodo multiplo, fino ad interpolare il punto di controllo corrispondente (quando $molt=h+1$)

perchè ?

PERCHE'?

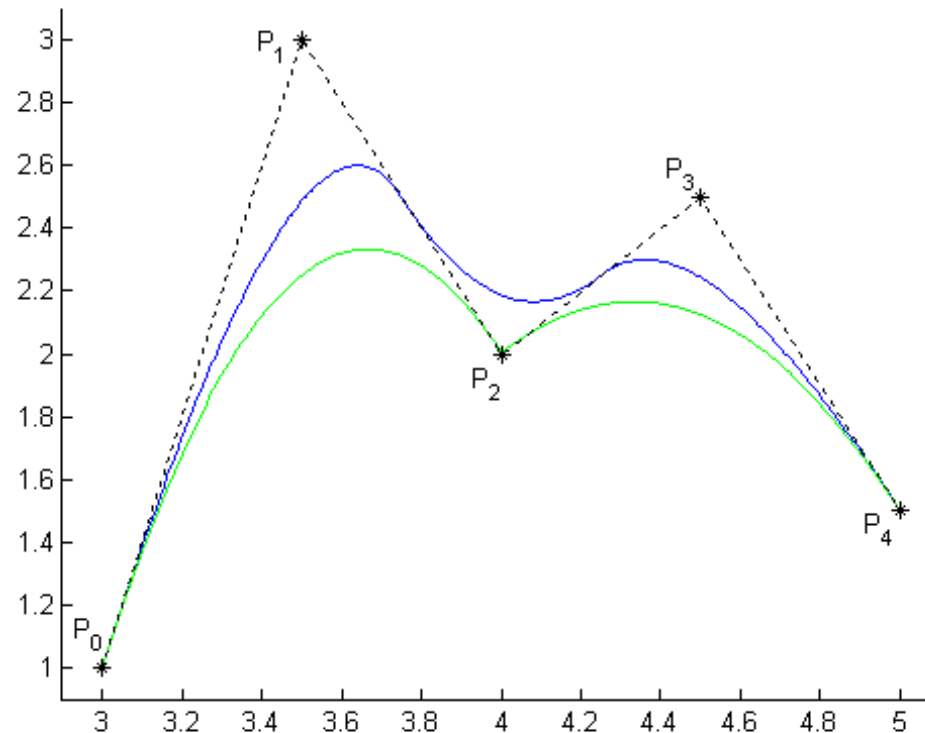
- Quanto più aumenta la molteplicità di un nodo t_i tanto più si riduce il supporto della funzione B-spline definita su quel nodo . Poiché la B-spline deve avere valori in $[0,1]$ il risultato è che tende sempre più rapidamente al suo valore massimo (1)



Nodi interni

$h=2$:

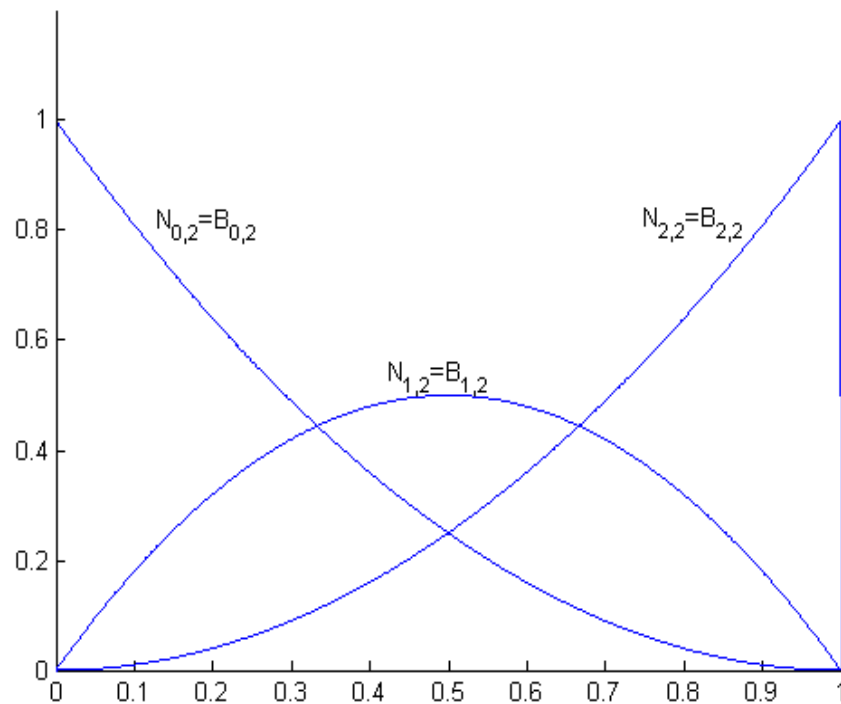
- $T=(\underline{0,0,0},\underline{1,2,3,3,3})$
 $C(t) \ t \in [0,3]$
- $T=(\underline{0,0,0},\underline{1,1,2,2,2})$
 $C(t) \ t \in [0,2]$



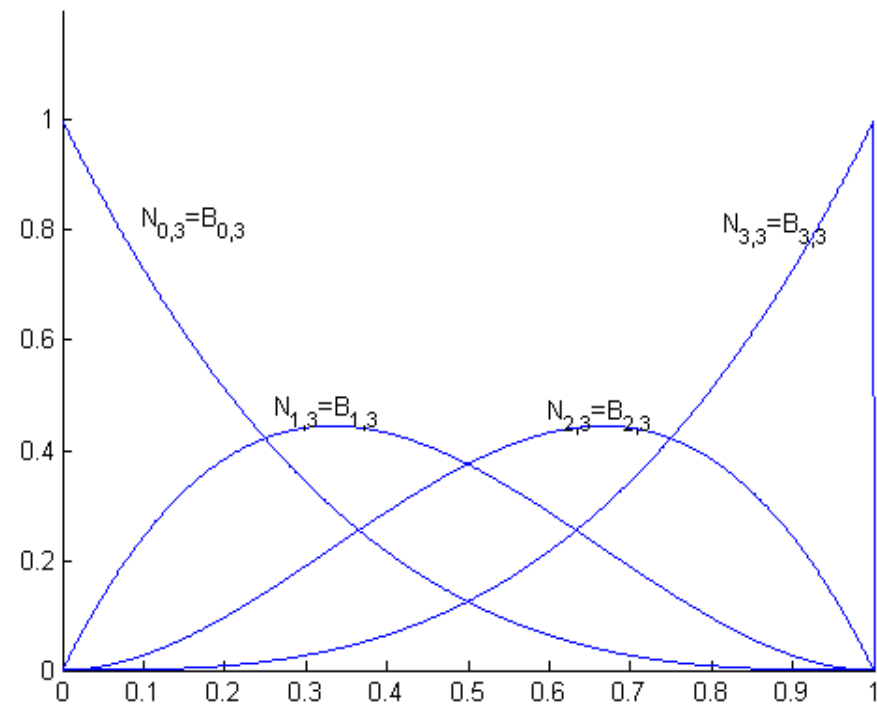
t_j nodo di molteplicità $k \Rightarrow N_{i,h}(t) \in C^{h-k}$

Molteplicità nelle Funzioni B-Spline: polinomi di Bernstein

$T=(0,0,0,1,1,1)$
grado = 2



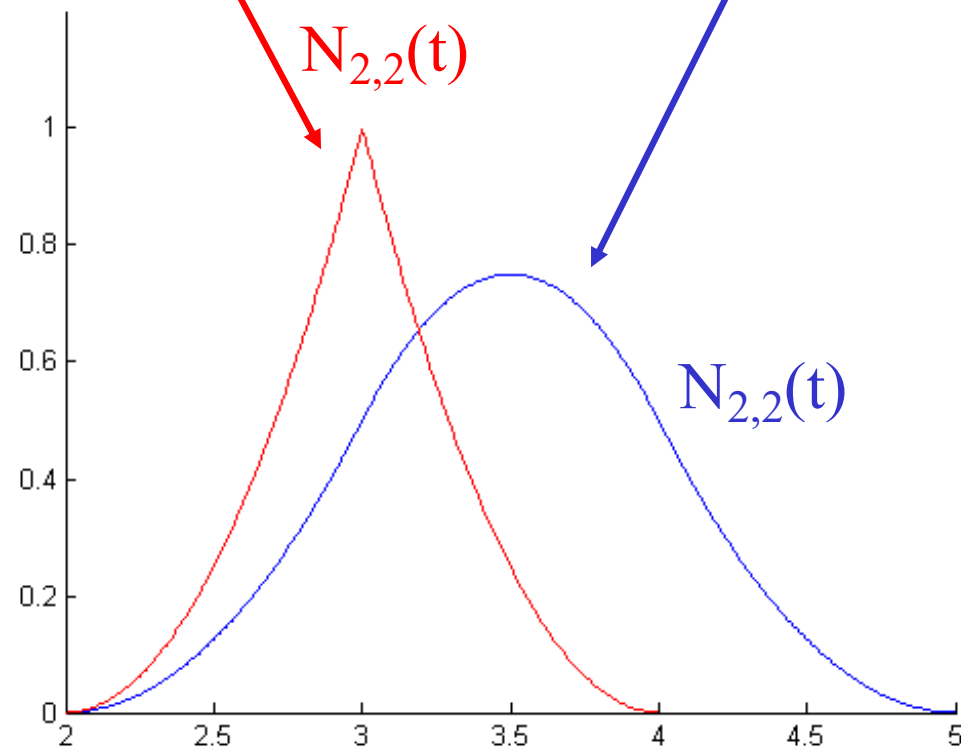
$T=(0,0,0,0,1,1,1,1)$
grado = 3



Molteplicità nelle Funzioni B-Spline: continuità

$T=(0,1,2,3,3,4,5,6)$
grado = 2

$T=(0,1,2,3,4,5,6,7)$
grado = 2



Proprietà delle funzioni di base B-Spline

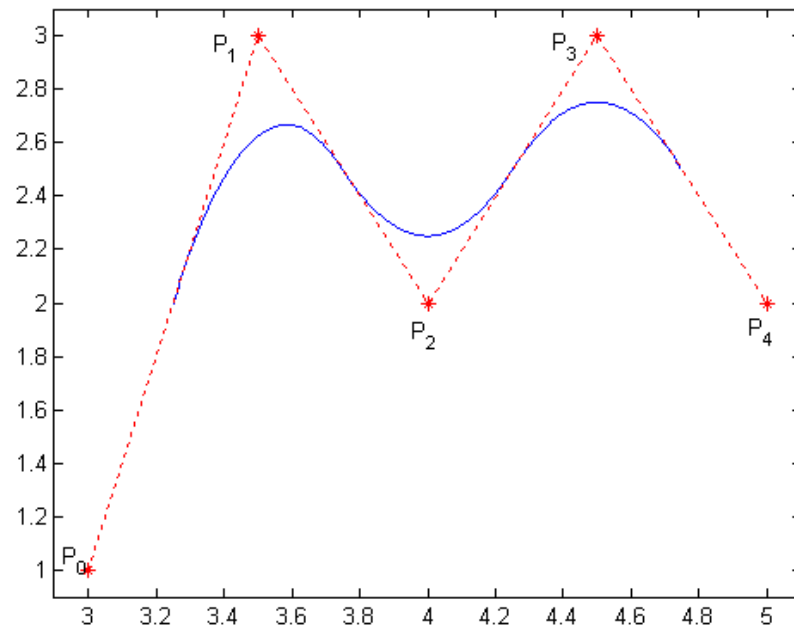
- $N_{i,h}(t)$ è un polinomio di grado h in t
- $t=t_j$ nodo di molteplicità $k \rightarrow N_{i,h}(t) \in C^{h-k}$
- *Non negatività:* $N_{i,h}(t) \geq 0 \quad \forall t$
- *Supporto locale:* $N_{i,h}(t) = 0 \quad \forall t \notin (t_i, t_{i+h+1}) \quad \forall i$

In $[t_j, t_{j+1})$ sono non nulle al più $N_{j-h,h}(t), \dots, N_{j,h}(t)$

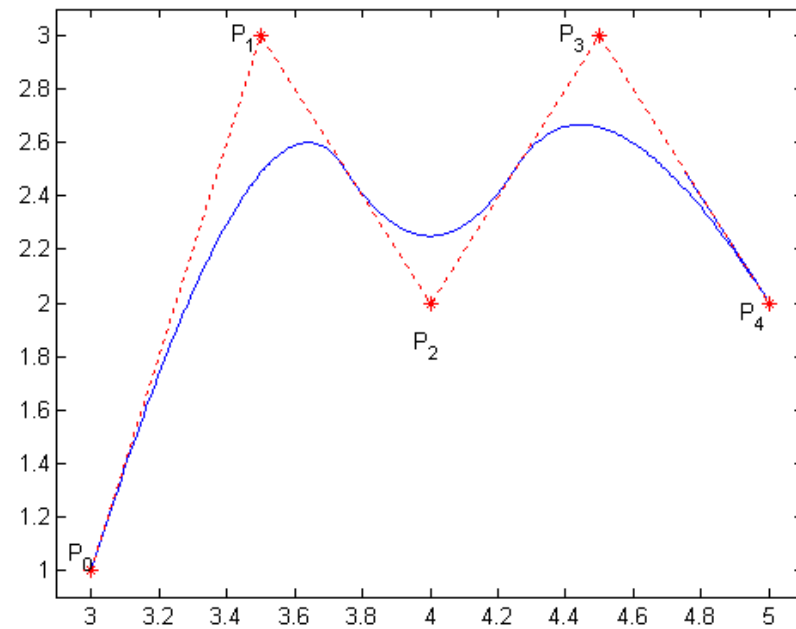
- *Partizione dell'unità:* $\sum_{i=0}^n N_{i,h}(t) = \sum_{i=j-h}^j N_{i,h}(t) = 1 \quad \forall t \in [t_j, t_{j+1})$
- $\{N_{0,h}(t), \dots, N_{n,h}(t)\}$ base per lo spazio delle curve spline di grado h sul vettore dei nodi T
- *Generalizzazione dei polinomi di Bernstein*

Esempio: curve B-Spline di grado 2

$T=(0, 1, 2, 3, 4, 5, 6, 7)$

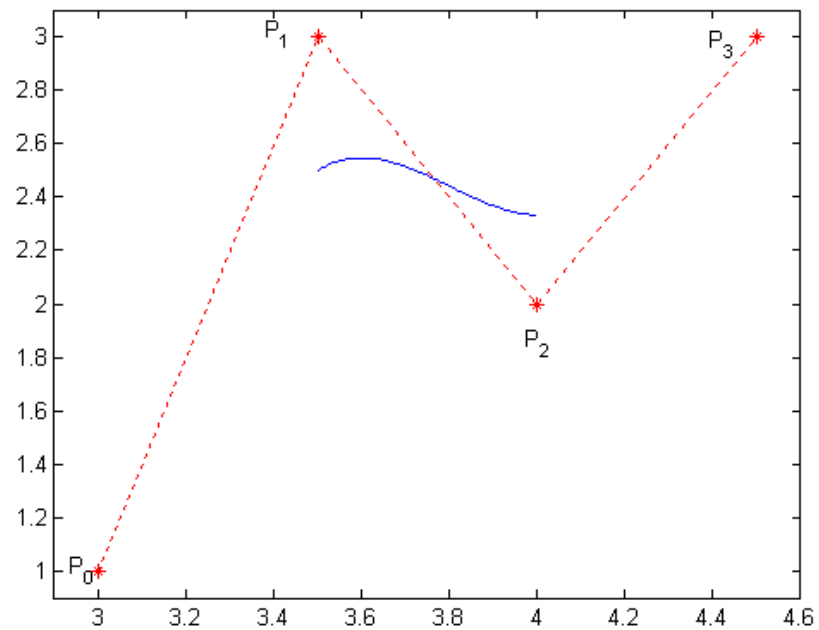


$T=(0, 0, 0, 1, 2, 3, 3, 3)$

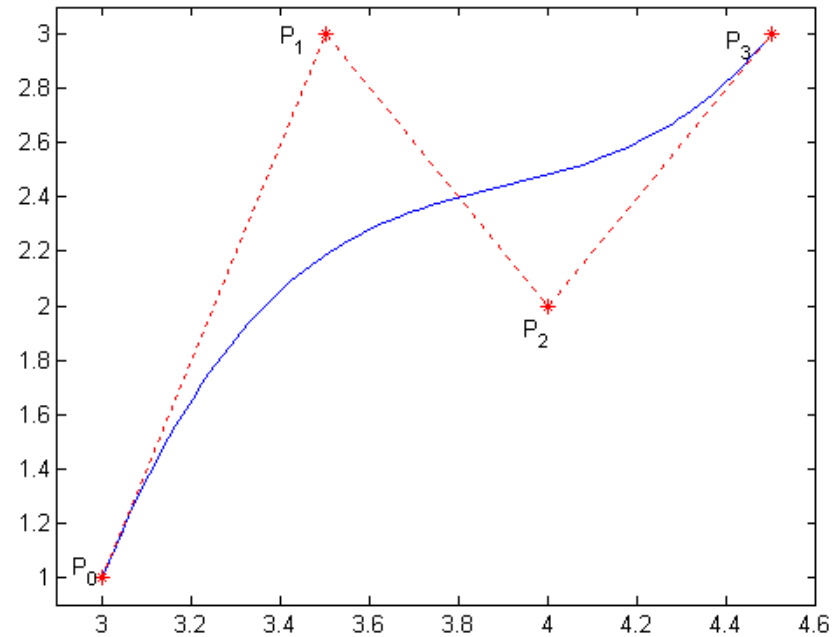


Esempio: curve B-Spline di grado 3

$T=(0, 1, 2, 3, 4, 5, 6, 7)$



$T=(0, 0, 0, 0, 1, 1, 1, 1)$

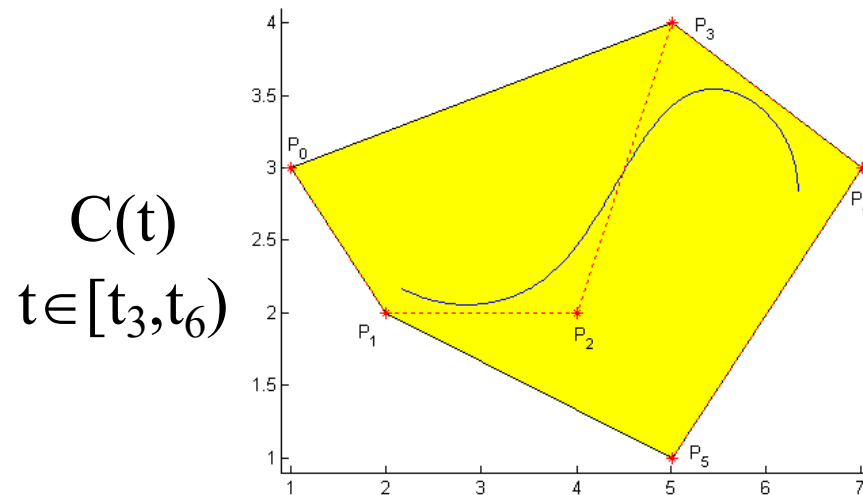


Proprietà delle curve B-Spline

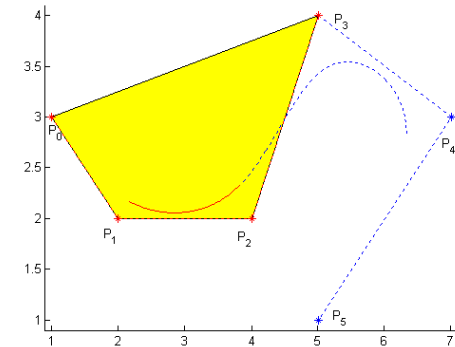
- *Interpolazione*: $C(t)$ in genere NON interpola P_0 e P_n
- *Variation diminishing*: nessuna retta (o piano) ha più intersezioni con la curva che con il poligono di controllo
- *Invarianza affine*: la relazione fra la curva e i punti di controllo è invariante per trasformazioni affini
- *Generalizzazione delle curve di Bezier*

Proprietà delle curve B-Spline (cont.)

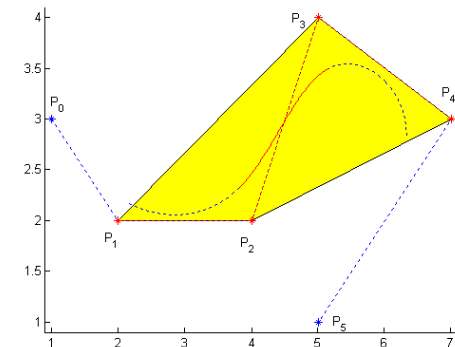
- *Convex hull “ristretto”*: giace completamente all’interno del convex hull dei punti di controllo che contribuiscono alla sua valutazione;



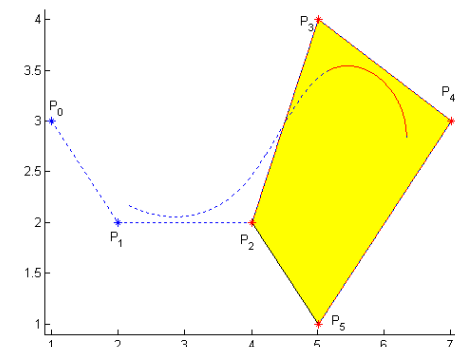
$C(t)$
 $t \in [t_3, t_4)$



$C(t)$
 $t \in [t_4, t_5)$



$C(t)$
 $t \in [t_5, t_6)$



$t \in [t_j, t_{j+1}) \longrightarrow C(t)$ giace nel c. h. di $P_{j-h}, \dots, P_j$

Proprietà delle curve B-Spline (cont.)

- *Controllo locale*: se P_j viene spostato, $C(t)$ risulta modificata solo in corrispondenza degli $h+1$ intervalli di nodi $[t_j, t_{j+h+1})$

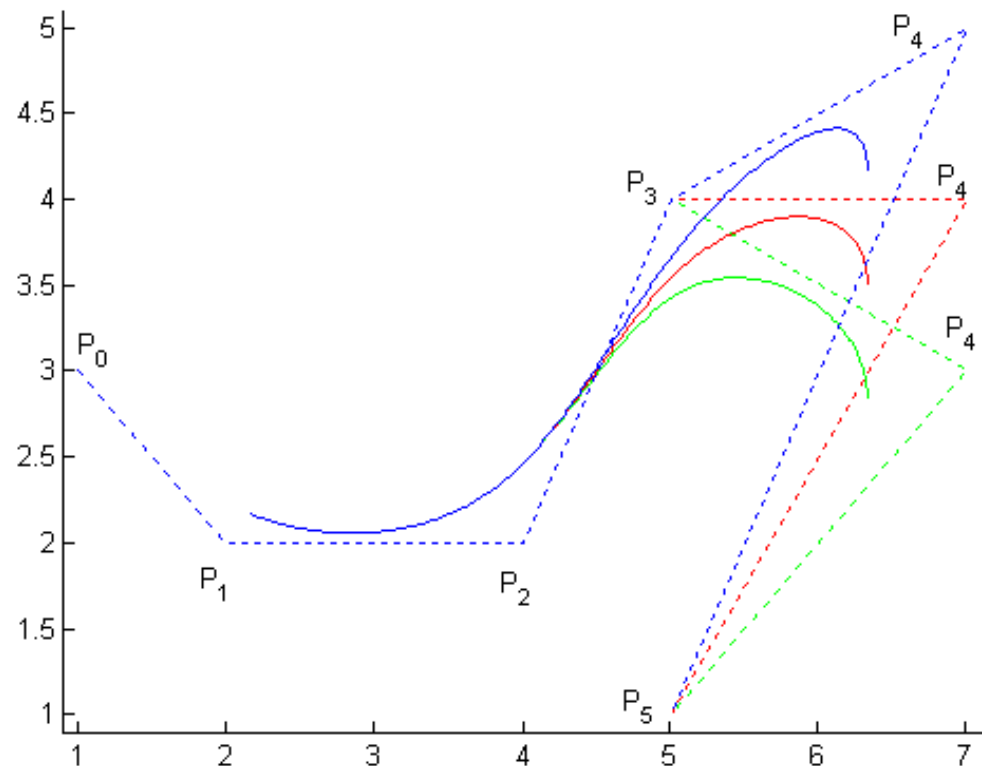
$h=3$:

$T=(0,1,2,3,4,5,6,7,8,9)$

$P_0, P_1, P_2, P_3, P_4, P_5,$

$C(t) \ t \in [3,6]$

Spostando P_4 , $C(t)$ si modifica solo per $t \in [t_4, t_8)$



Proprietà delle curve B-Spline (cont.)

- $t \neq t_i \quad \forall i \rightarrow C(t) \in C^\infty$
- La *derivata prima* della curva B-Spline di grado h :

$$C(t) = \sum_{i=0}^n P_i N_{i,h}(t)$$

è una curva B-Spline di grado $h-1$:

$$C'(t) = \frac{d}{dt} C(t) = n \sum_{i=0}^{n-1} Q_i N_{i+1,h-1}(t)$$

definita sullo stesso vettore dei nodi $T=(t_0, t_1, \dots, t_{n+h+1})$,
con punti di controllo:

$$Q_i = h \frac{P_{i+1} - P_i}{t_{i+h+1} - t_{i+1}}, \quad i = 0, \dots, n-1$$

Parametri di forma di una curva B-Spline

Principali tecniche per modificare la forma:

- Scelta del *vettore dei nodi* $T=(t_0, t_1, \dots, t_{n+h+1})$
(uniforme vs. non uniforme)
- Scelta del *grado* h
- Scelta dei *punti di controllo* $P_0, \dots, P_n$

Vettore dei nodi uniforme

- Al crescere del grado h , la curva si allontana dal poligono di controllo e diventa “più corta”

$h=2$:

$T=(0,1,2,3,4,5,6,7,8)$

$C_2(t) \ t \in [2,6]$

$h=3$:

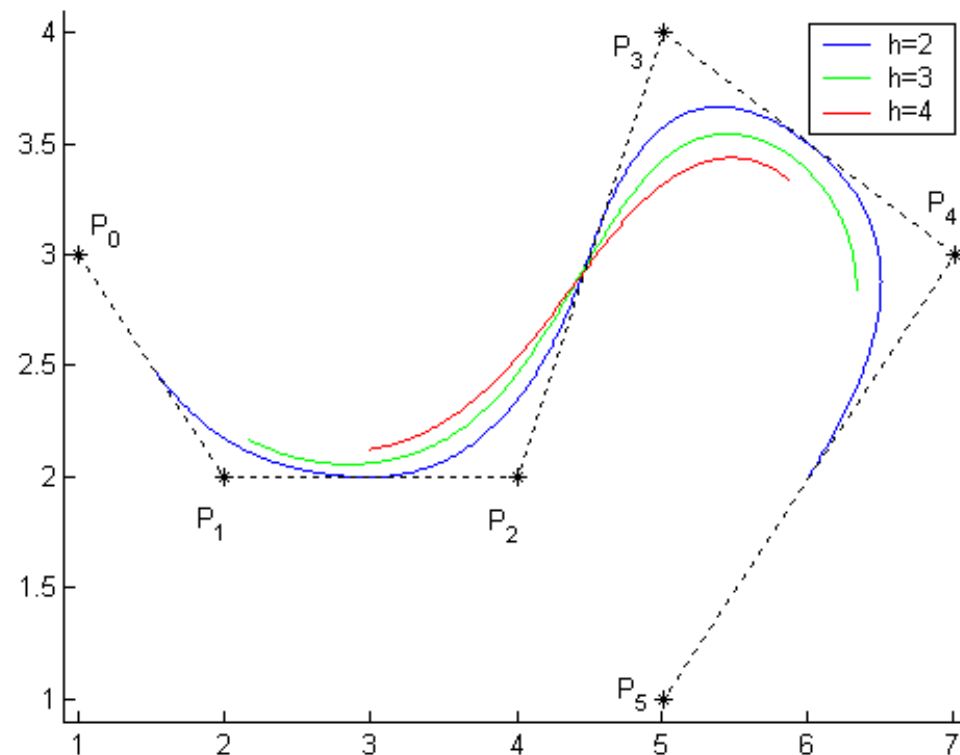
$T=(0,1,2,3,4,5,6,7,8,9)$

$C_3(t) \ t \in [3,6]$

$h=4$:

$T=(0,1,2,3,4,5,6,7,8,9,10)$

$C_4(t) \ t \in [4,6]$

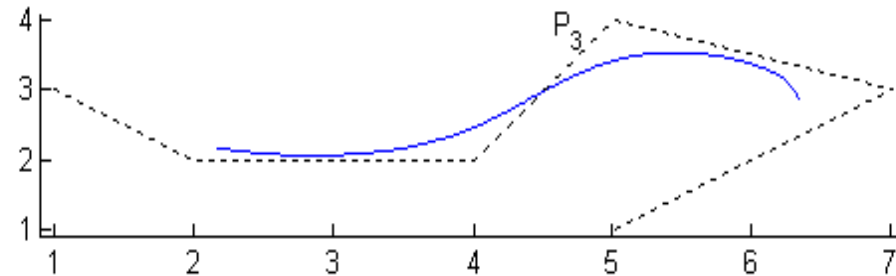


Vettore dei nodi uniforme

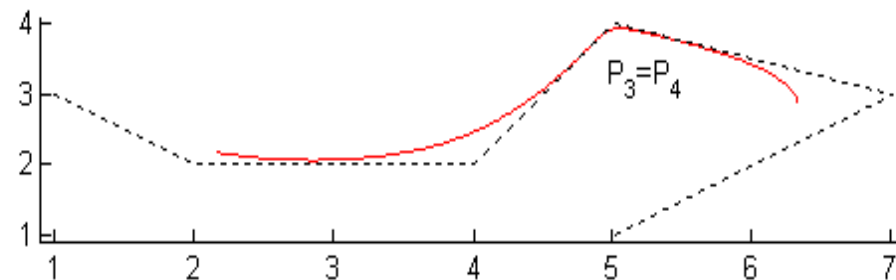
- **All'aumentare della molteplicità di un punto di controllo, la curva si avvicina al punto multiplo, fino ad interpolarlo (quando $molt=h$)**

$h=3$:

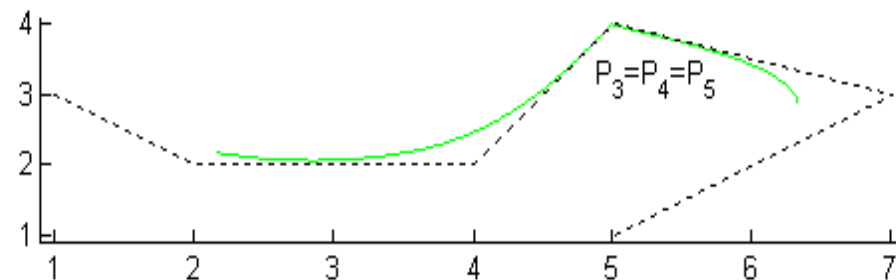
- $T=(0,1,2,3,4,5,6,7,8,9)$
 $P_0, P_1, P_2, \mathbf{P_3}, P_4, P_5,$
 $C(t) \ t \in [3,6]$



- $T=(0,1,2,3,4,5,6,7,8,9,10)$
 $P_0, P_1, P_2, \mathbf{P_3=P_4}, P_5, P_6,$
 $C(t) \ t \in [3,7]$



- $T=(0,1,2,3,4,5,6,7,8,9,10,11)$
 $P_0, P_1, P_2, \mathbf{P_3=P_4=P_5}, P_6,$
 $C(t) \ t \in [3,8]$



Vettore dei nodi non uniforme

Nodi estremi con molteplicità pari all'ordine $(h+1)$ della curva:

- **Al crescere del grado h** , la curva si allontana dal poligono di controllo

$h=2$:

$$T=(0,0,0,1,2,3,3,3)$$

$$C_2(t) \quad t \in [0,3]$$

$h=3$:

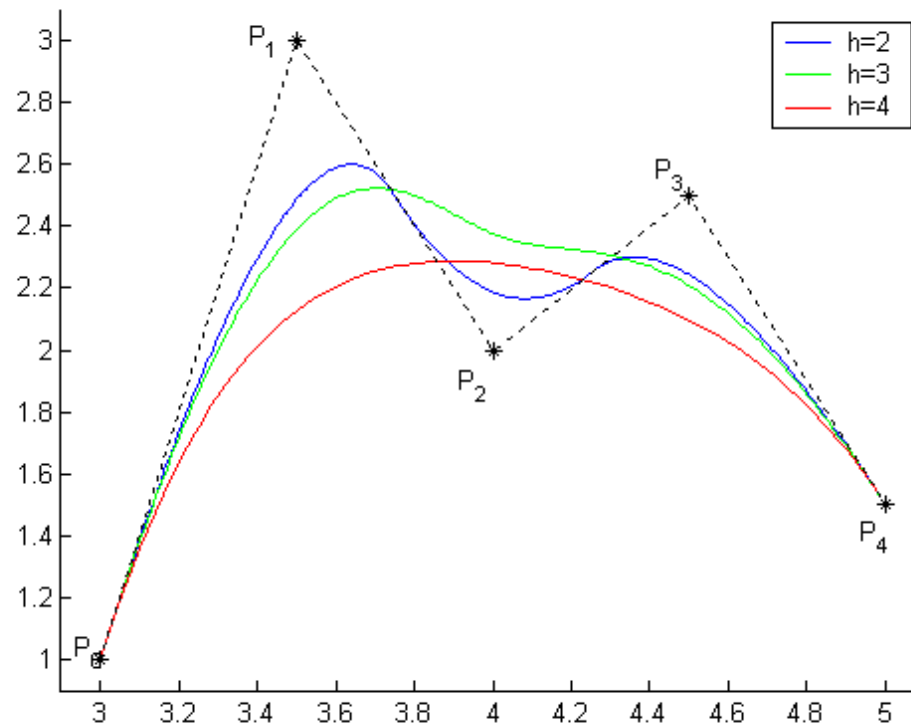
$$T=(0,0,0,0,1,2,2,2,2)$$

$$C_3(t) \quad t \in [0,2]$$

$h=4$:

$$T=(0,0,0,0,0,1,1,1,1,1)$$

$$C_4(t) \quad t \in [0,1]$$



- **Punti di controllo multipli:** comportamento analogo al caso del vettore dei nodi uniforme

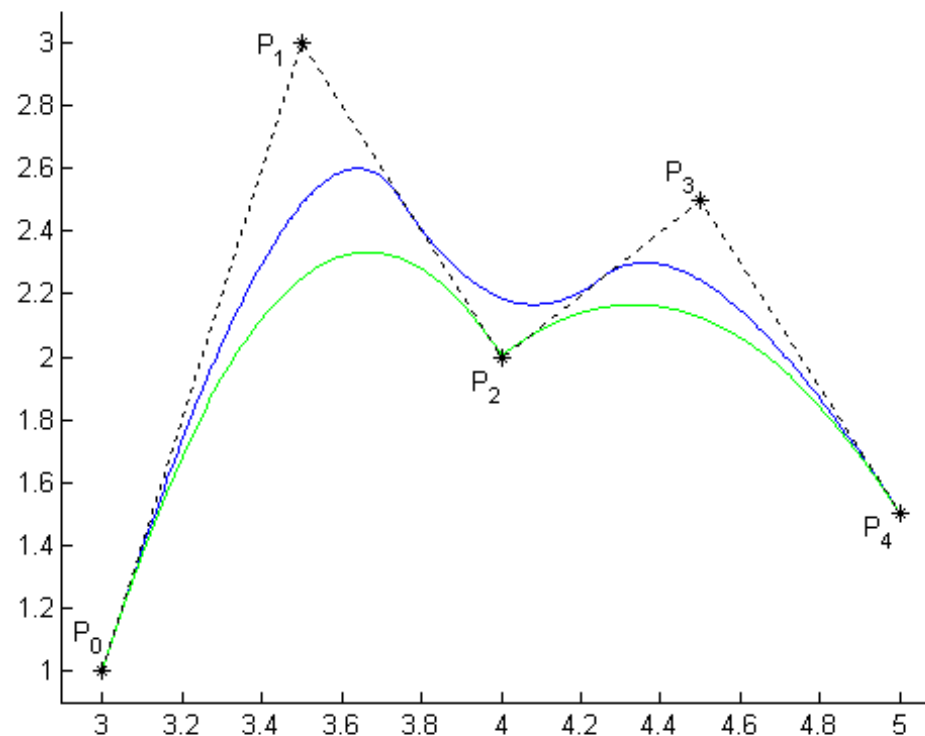
Vettore dei nodi non uniforme

Nodi interni multipli:

- **All'aumentare della molteplicità del nodo, la curva si avvicina al poligono di controllo**

$h=2$:

- $T=(0,0,0,1,2,3,3,3)$
 $C(t) \ t \in [0,3]$
- $T=(0,0,0,1,1,2,2,2)$
 $C(t) \ t \in [0,2]$



Valutazione di una curva B-Spline

Assegnati:

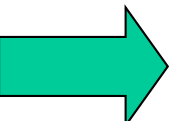
- il grado h
- i punti di controllo $P_0, \dots, P_n$
- il vettore dei nodi $T=(t_0, \dots, t_{n+h+1})$

valutare per un generico valore t del parametro:

$$C(t) = \sum_{i=0}^n P_i N_{i,h}(t)$$

I passo

Individuare:

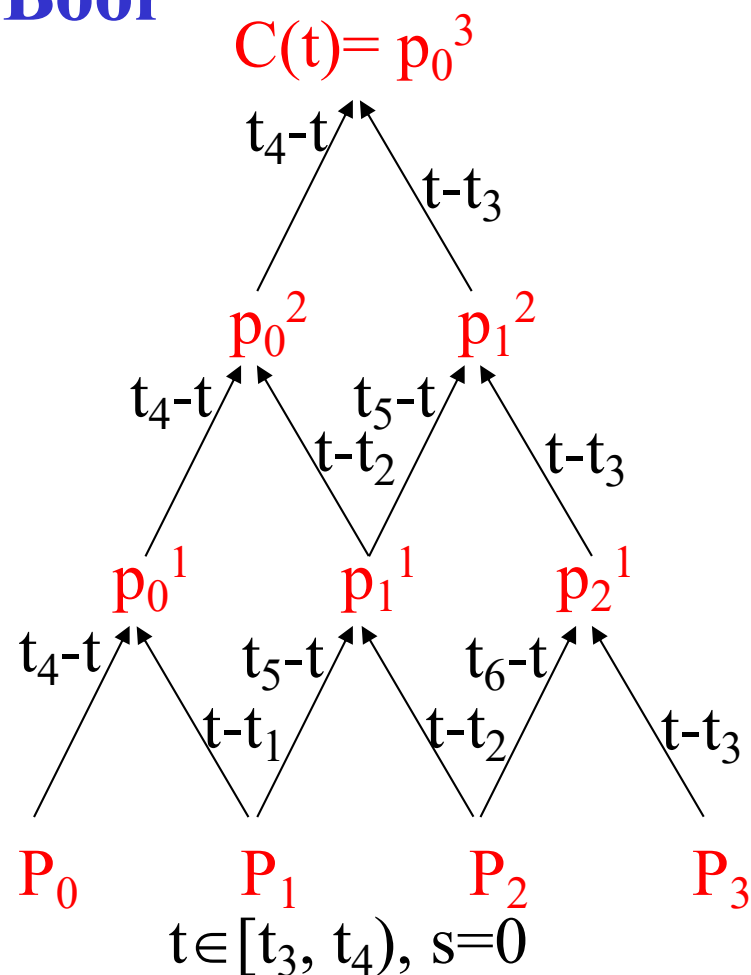
- j tale che $t \in [t_j, t_{j+1})$  $C(t) = \sum_{i=j-h}^j P_i N_{i,h}(t) \quad t \in [t_j, t_{j+1})$
- s = molteplicità di t_j in T

Valutazione di una curva B-Spline (cont.)

Il passo

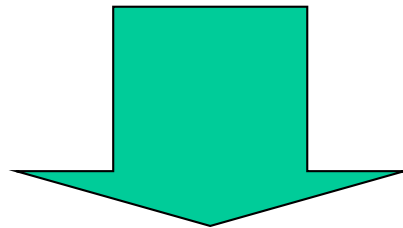
Algoritmo di de Boor

```
for i=j-h, j
   $p_i^0 = P_i$ 
endfor
for r=1, h-s
  for i=j-h, j-r
     $a_i^r = (t - t_{i+r}) / (t_{i+h+1} - t_{i+r})$ 
     $p_i^r = (1 - a_i^r) p_i^{r-1} + a_i^r p_{i+1}^{r-1}$ 
  endfor
endfor
Risultato:  $C(t) = p_{j-h}^{h-s}$ 
 $T_{DB}(h) = O(3/2h^2)$ 
```



Svantaggio delle curve B-Spline

Non consentono di rappresentare esattamente alcune forme comuni, quali le sezioni coniche (circonferenza, ellisse, parabola, iperbole)



Curve NURBS

Curve NURBS (Non Uniform Rational B-Spline)

- $P_0, \dots, P_n$ punti di controllo, $w_0, \dots, w_n$ pesi (≥ 0)

$$C(t) = \frac{\sum_{i=0}^n w_i P_i N_{i,h}(t)}{\sum_{i=0}^n w_i N_{i,h}(t)} \quad t \in [t_h, t_{n+h+1}]$$

con $N_{i,h}(t)$ *funzioni di base B-spline* di grado h sul
vettore dei nodi $T=(t_0, t_1, \dots, t_{n+h+1})$

Funzioni B-Spline Razionali

- Una curva NURBS può essere riscritta nella forma equivalente:

$$C(t) = \sum_{i=0}^n P_i R_{i,h}(t) \quad t \in [t_h, t_{n+h+1}]$$

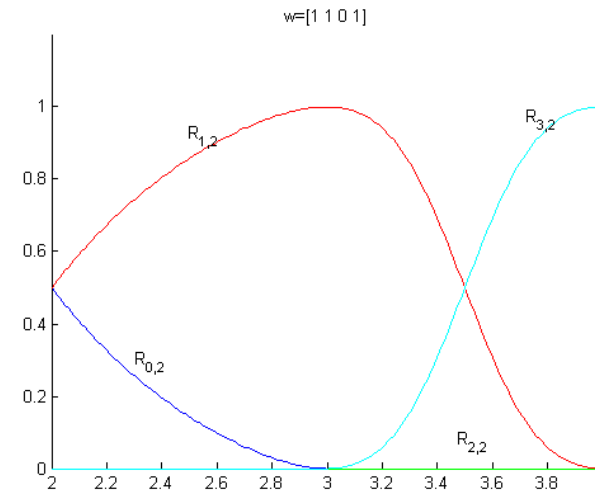
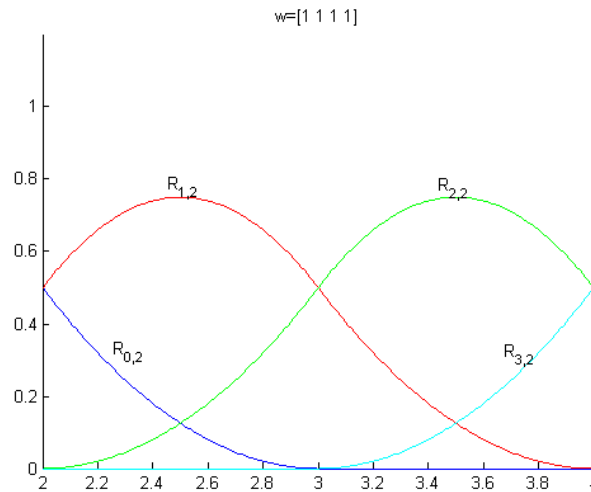
con $R_{i,h}(t)$ *funzioni B-spline razionali* di grado h con vettore dei nodi $T=(t_0, t_1, \dots, t_{n+h+1})$ e pesi $w_0, \dots, w_n$:

$$R_{i,h}(t) = \frac{w_i N_{i,h}(t)}{\sum_{i=0}^n w_i N_{i,h}(t)} \quad t \in [t_h, t_{n+h+1}]$$

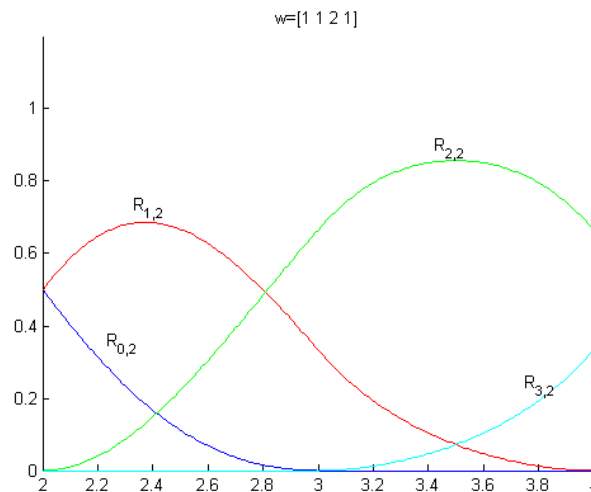
Funzioni B-Spline Razionali: esempi

- Grado $h=2$, $T=(0,1,2,3,4,5,6)$, $w_0=w_1=w_3=1$

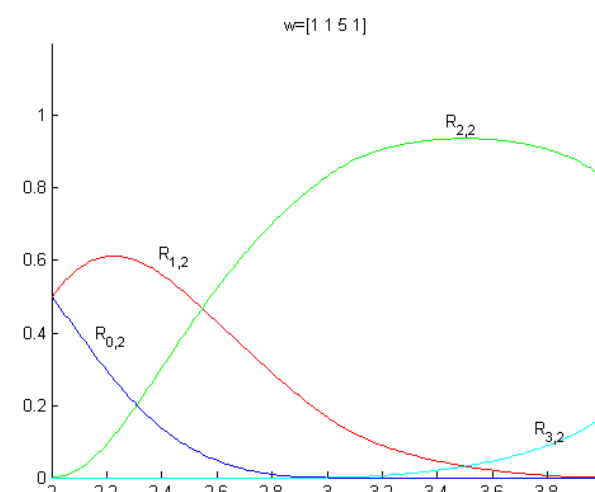
$w_2=1$
 $R_{i,h}(t)$
 $\parallel$
 $N_{i,h}(t)$



$w_2=0$



$w_2=2$



$w_2=5$

Proprietà delle funzioni B-Spline razionali

Analoghe a quelle delle funzioni di base B-spline:

- $R_{i,h}(t)$ è un polinomio razionale di grado h in t
- $t=t_j$ nodo di molteplicità $k \rightarrow R_{i,h}(t) \in C^{h-k}$
- *Non negatività:* $R_{i,h}(t) \geq 0 \quad \forall t$
- *Supporto locale:* $R_{i,h}(t) = 0 \quad \forall t \notin (t_i, t_{i+h+1}) \quad \forall i$

In $[t_j, t_{j+1})$ sono non nulle al più $R_{j-h,h}(t), \dots, R_{j,h}(t)$

- *Partizione dell'unità:* $\sum_{i=0}^n R_{i,h}(t) = \sum_{i=j-h}^j R_{i,h}(t) = 1 \quad \forall t \in [t_j, t_{j+1})$
- *Generalizzazione dei polinomi di Bernstein e delle funzioni B-Spline*

Proprietà delle curve NURBS

Analoghe a quelle delle curve B-Spline:

- *Interpolazione*: $C(t)$ in genere NON interpola P_0 e P_n
- *Variation diminishing*: nessuna retta (o piano) ha più intersezioni con la curva che con il poligono di controllo
- *Generalizzazione delle curve di Bezier* (e B-Spline)
- *Convex hull “ristretto”*: se $t \in [t_j, t_{j+1})$, $C(t)$ giace nel convex hull dei punti di controllo $P_{j-h}, \dots, P_j$
- $t \neq t_i \forall i \rightarrow C(t) \in C^\infty$
- $t = t_i$ nodo di molteplicità $k \rightarrow C(t) \in C^{h-k}$

Proprietà delle curve NURBS (cont.)

Inoltre:

- *Controllo locale*: se P_j viene spostato o w_j è modificato, $C(t)$ risulta modificata solo in corrispondenza degli $h+1$ intervalli di nodi $[t_j, t_{j+h+1})$
- *Invarianza affine e proiettiva*: la relazione fra la curva e i punti di controllo è invariante per trasformazioni affini e proiettive

Parametri di forma di una curva NURBS

Come per le curve B-spline:

- Scelta del *vettore dei nodi* $T=(t_0, t_1, \dots, t_{n+h+1})$ (uniforme vs. non uniforme)
- Scelta del *grado* h
- Scelta dei *punti di controllo* $P_0, \dots, P_n$

Inoltre:

- Scelta dei pesi $w_0, \dots, w_n$

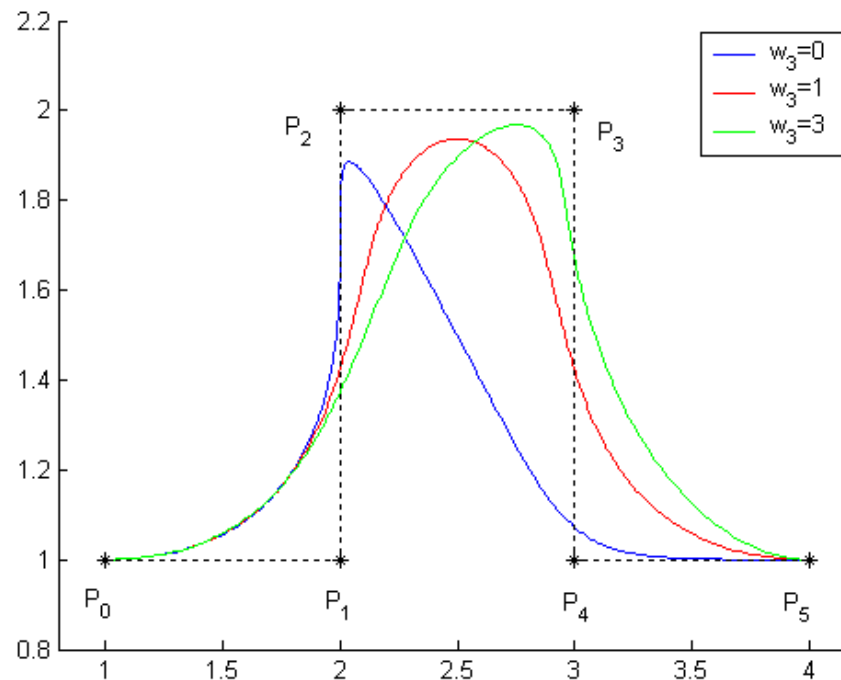
Scelta dei pesi

- **All'aumentare del valore di un peso, la curva si avvicina al corrispondente punto di controllo**

$h=3$:

$T=(0,0,0,1,2,3,3,3)$

- $W=(1,1,1,0,1,1)$
- $W=(1,1,1,1,1,1)$
- $W=(1,1,1,3,1,1)$



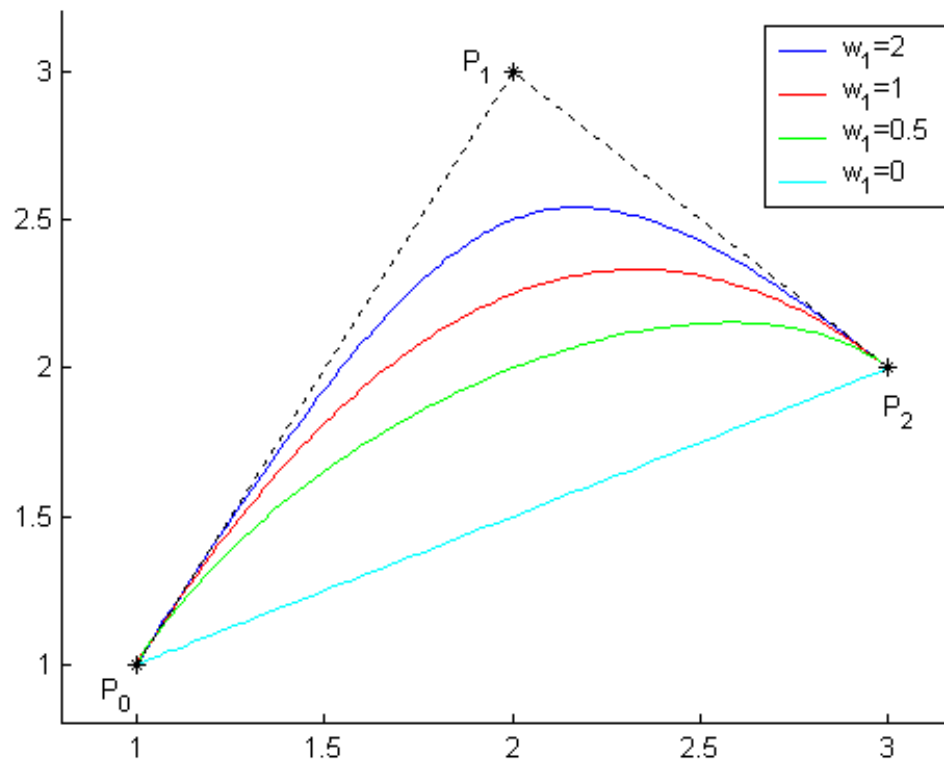
Curve NURBS e sezioni coniche

- Qualsiasi sezione conica può essere rappresentata mediante una curva NURBS di grado 2 con vettore dei nodi $T=(0,0,0,1,1,1)$ (*curve di Bezier razionali*)

$h=2$

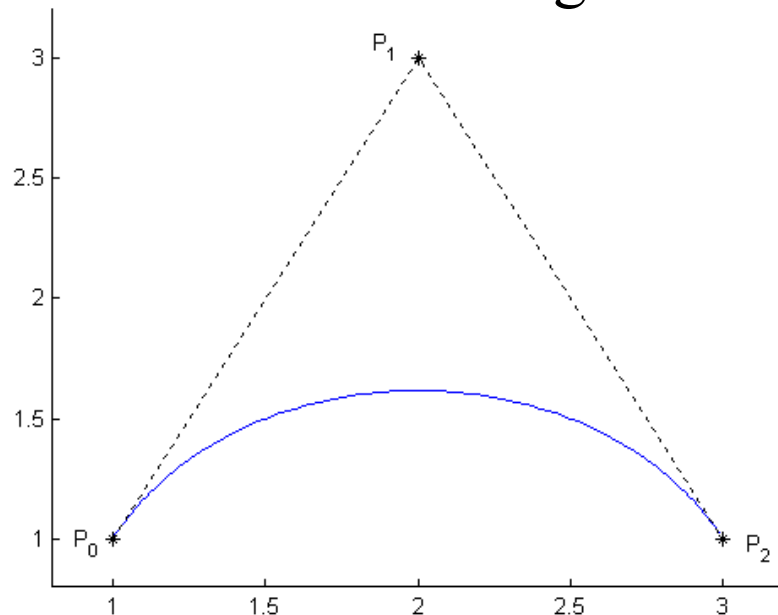
$w_0 = w_2 = 1$

- $w_1 > 1$: iperbole
- $w_1 = 1$: parabola
- $w_1 < 1$: ellisse
- [$w_1 = 0$: retta]

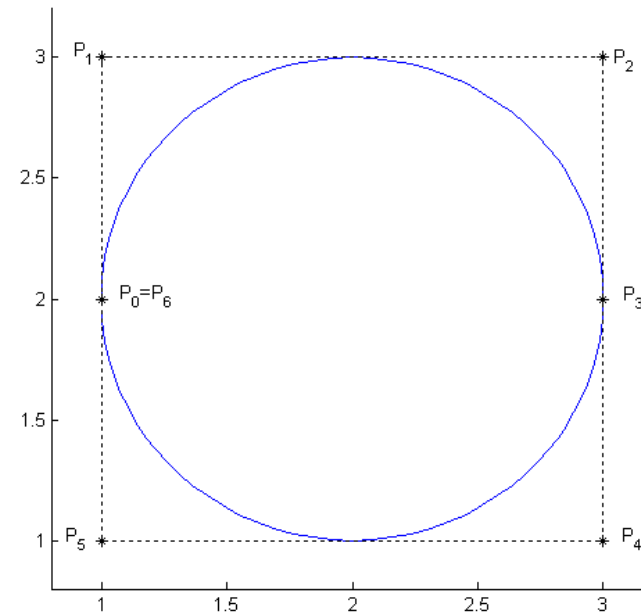


Curve NURBS e sezioni coniche (cont.)

- Una circonferenza può essere rappresentata mediante curve NURBS di grado 2



$$T=(0,0,0,1,1,1),$$
$$w=(1,1/\sqrt{5},1)$$



$$T=(0,0,0,0.25,0.5,0.5,0.75,1,1,1),$$
$$w=(1,0.5,0.5,1,0.5,0.5,1)$$

(4 archi di circonferenza che
sottendono un angolo di 90°)

Curve NURBS e coordinate omogenee

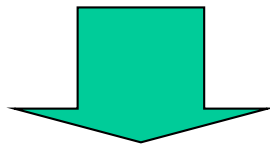
- Un modo alternativo di definire la curva NURBS:

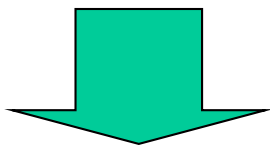
$$C(t) = \sum_{i=0}^n P_i R_{i,h}(t) \quad t \in [t_h, t_{n+h+1}]$$

è come proiezione su $\mathcal{R}^3$ di una curva B-Spline in $\mathcal{R}^4$ avente come punti di controllo i punti $P_i = [x_i, y_i, z_i]^T$ espressi in coordinate omogenee:

$$P_i^w = [x_i w_i, y_i w_i, z_i w_i, w_i]^T \quad i = 0, \dots, n$$

$$C(t) = H \left(\sum_{i=0}^n P_i^w N_{i,h}(t) \right) = \sum_{i=0}^n H(P_i^w) R_{i,h}(t) \quad t \in [t_h, t_{n+h+1}]$$





Strategia generale per algoritmi relativi alle curve NURBS

1. Calcolo delle coordinate omogenee P_i^W dei punti di controllo P_i
2. Applicazione dell'algoritmo alla curva B-Spline con punti di controllo P_i^W
3. Proiezione (omogeneizzazione) del risultato

Curve B-Spline e NURBS in OpenGL

GLU fornisce un insieme di funzioni per la modellazione ed il rendering di curve B-Spline e NURBS, basate sui valutatori di OpenGL:

gluNewNurbsRenderer() restituisce un puntatore ad un oggetto NURBS

gluBeginCurve() per iniziare la curva

gluNurbsCurve() per specificare vettore dei nodi, punti di controllo e ordine della curva (almeno una volta)

gluEndCurve() per terminare la curva

gluNurbsProperty() sceglie valori di rendering (es. max dimensione delle linee usate per il rendering degli oggetti NURBS)

gluNurbsCallback() per la notifica di errori

Curve B-Spline in OpenGL: esempio

```
GLfloat cp[4][3] = {      { -4.0, -4.0, 0.0}, { -2.0,  4.0, 0.0},  
                        {  2.0, -4.0, 0.0}, {  4.0,  4.0, 0.0}};
```

```
GLUnurbsObj *theNurb;
```

```
GLfloat knots[8] = {0.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 1.0};
```

```
...
```

```
theNurb=gluNewNurbsRenderer(); /*puntatore all'oggetto NURBS */
```

```
...
```

```
gluBeginCurve(theNurb); /*inizia la curva*/
```

```
gluNurbsCurve(theNurb, 8, knots, 3, &cp[0][0] 4 GL_MAP1_VERTEX_3);
```



```
/* vettore dei nodi knots, punti di controllo cp e ordine 4 */
```

```
gluEndCurve(theNurb) /* termina la curva*/
```

Curve NURBS in OpenGL: esempio

```
GLfloat cp[4][3] = {      { -4.0, -4.0, 0.0}, { -2.0,  4.0, 0.0},  
                        {  2.0, -4.0, 0.0}, {  4.0,  4.0, 0.0}};
```

```
GLUnurbsObj *theNurb;
```

```
GLfloat knots[8] = {0.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 1.0};
```

```
GLfloat w[4] = { 1.0, 5.0, 1.0, 1.0};
```

```
GLfloat cpw[4][4];
```

```
...
```

“Calcolo cpw (coordinate omogenee dei cp)”

```
...
```

```
theNurb=gluNewNurbsRenderer(); /*puntatore all'oggetto NURBS */
```

```
...
```

```
gluBeginCurve(theNurb); /*inizia la curva*/
```

```
gluNurbsCurve(theNurb, 8, knots, 4, &cpw[0][0], 4, GL_MAP1_VERTEX_4);
```

```
/* vettore dei nodi knots, punti di controllo in coord. omog. cpw, ordine 4 */
```

```
gluEndCurve(theNurb) /* termina la curva*/
```

Proprietà delle curve NURBS in OpenGL

gluNurbsProperty(GLUnurbsObj *nobj,
Glenum property, Gfloat value)

sceglie opportuni valori per il rendering

Esempio:

theNurb=gluNewNurbsRenderer();

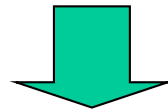
gluNurbsProperty(theNurb, GLU_U_STEP, 5);

gluNurbsProperty(theNurb, GLU_SAMPLING_METHOD,
GLU_DOMAIN_DISTANCE);

impone il campionamento della curva in GLU_U_STEP=5 valori equispaziati del parametro

NURBS in OpenGL: gestione degli errori

Sono previsti 37 differenti errori specifici per le funzioni di GLU
relativi alle NURBS



```
gluNurbsCallback( GLUnurbsObj *nobj,  
                  GLenum tipo,  
                  void (*fn)(GLenum errorCode);
```

- **tipo** = GLU_ERROR
- **fn** = function richiamata quando si verifica un errore
- **errorCode** = argomento passato alla function fn

Gestione degli errori NURBS: esempio

```
GLvoid nurbsError(GLenum errorCode) /* definita dall'utente */
{
    const GLubyte *estring;

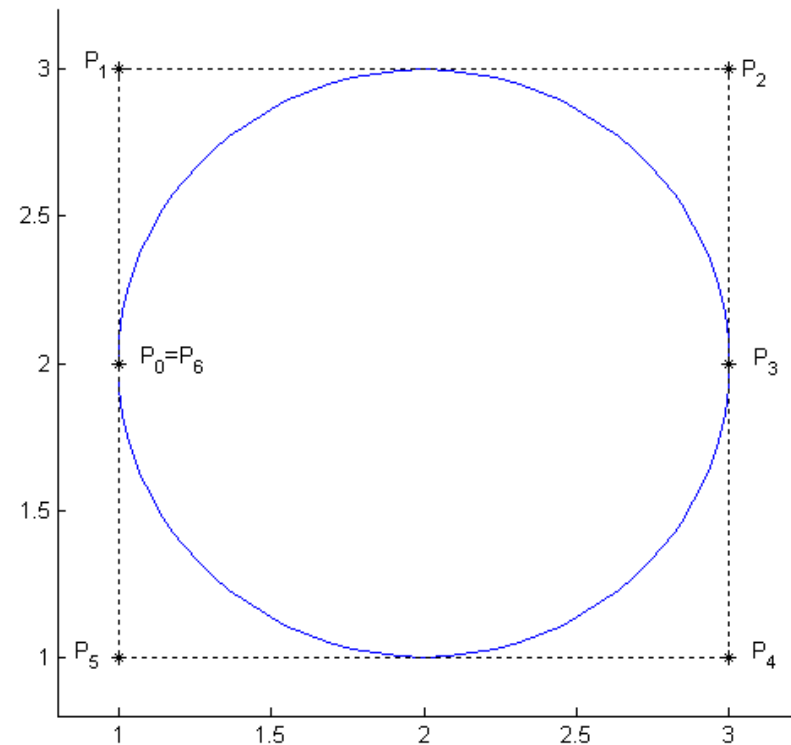
    estring = gluErrorString(errorCode);
    fprintf (stderr, "Errore NURBS: %s\n", estring);
    exit (0);
}

...
theNurb = gluNewNurbsRenderer();
gluNurbsCallback(theNurb, GLU_ERROR, (GLvoid (*)(GLvoid))nurbsError);
...
```

Esercizio

Scrivere un programma con OpenGL che:

- disegni una circonferenza mediante curve NURBS
- campioni la curva per un prefissato numero di valori del parametro
- gestisca gli eventuali errori



Esercizio

Scrivere un programma con OpenGL che:

- disegni una circonferenza mediante curve NURBS
- campioni la curva per un prefissato numero di valori del parametro
- gestisca gli eventuali errori

$$T=(0,0,0,0.25,0.5,0.5,0.75,1,1,1)$$
$$w=(1,0.5,0.5,1,0.5,0.5,1)$$

